МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) «Компьютерные науки и прикладная математика» **Кафедра** 406

Группа М8О-407Б-22 **Направление подготовки** 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Отказоустойчивая распределённая платформа мультиканальной гарантированной доставки уведомлений в микросервисной архитектуре

Автор ВКРБ: Батоян Роберт Левонович (_____)

Руководитель: Булакина Мария Борисовна (_____)

Консультант: Филимонов Николай Сергеевич (_____)

Консультант: _____ (_____)

Рецензент: _____ (_____)

К защите допустить

Заведующий кафедрой Кафедра 406 «Вычислительная математика и программирование» _ ( )

____ июня 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 77 страниц, 7 рисунков, 16 таблиц, 29 использованных источников, 5 приложений.

распределённые системы, микросервисная архитектура, уведомления, Kafka, gRPC, PostgreSQL, Redis, transactional outbox, transactional inbox, идемпотентность, отказоустойчивость

Выпускная квалификационная работа посвящена проектированию и реализации отказоустойчивой распределённой платформы доставки уведомлений в микросервисной архитектуре. В основе решения лежит декомпозиция уведомительного процесса на четыре взаимосвязанные стадии: приём команды на создание уведомительного события, формирование запуска доставки, исполнение канальной доставки и накопление истории статусов.

Цель работы заключается в построении архитектуры, обеспечивающей практическую гарантию обработки уведомлений в условиях асинхронного обмена сообщениями. Для достижения цели используются модель доставки не менее одного раза, ключи идемпотентности, дедупликация, транзакционный выходящий журнал, транзакционный входящий журнал, управляемые повторные попытки и наблюдаемость основных этапов обработки.

В работе исследованы требования к платформам уведомлений, обоснован выбор гибридной схемы взаимодействия, в которой синхронные команды выполняются по gRPC, а массовая фоновая обработка выполняется через Apache Kafka. Реализованы сервисы notification-facade, template-registry, profile-consent, scheduler-delivery, notification-mail-sender, notification-sms-sender и history-writer. Хранение доменных состояний и технических журналов организовано в PostgreSQL, а оперативный профиль получателя хранится в Redis и предоставляется через выделенный gRPC-сервис.

Практический результат работы представляет собой программную платформу, поддерживающую email- и SMS-доставку, отложенную отправку, предварительный рендеринг шаблонов, централизованную проверку согласий и ограничений получателя, а также агрегацию истории доставки. Архитектура и модель данных допускают дальнейшее расширение на новые каналы связи без переработки ядра событийной обработки.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 7
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ 8
ВВЕДЕНИЕ 9
1 Аналитическая глава, постановка задачи и требования 14
1.1 Платформа уведомлений как прикладная распределённая система 14
1.2 Проблемы прямой синхронной схемы доставки 14
1.3 Доменная модель уведомительного процесса 15
1.4 Архитектурные варианты и обоснование выбора 16
1.5 Сравнительный анализ транспортов межсервисного взаимодействия 16
1.6 Варианты работы с профилем получателя 17
1.7 Постановка задачи 17
1.7.1 Функциональные требования 18
1.7.2 Нефункциональные требования 18
1.7.3 Требования к надёжности и обработке ошибок 18
1.7.4 Требования к данным и хранению 19
1.8 Типовые сценарии использования 19
1.9 Выводы по разделу 20
2 Проектирование архитектуры распределённой платформы 23
2.1 Архитектурные принципы 23
2.2 Контейнерная декомпозиция 24
2.3 Границы ответственности и модель данных верхнего уровня . . 24
2.4 Поток данных при создании события 24
2.5 Поток данных при запуске доставки 25
2.6 Поток данных от планировщика к канальному сервису 26
2.7 Поток данных в канальных сервисах 27
2.7.1 Этап транспортного приёма 27
2.7.2 Этап планирования доставки 27
2.7.3 Этап фактической отправки 27
2.8 Архитектура обмена через Kafka 28
2.9 Границы транзакций и механизмы надёжности 28
2.9.1 Выходящий журнал 28
2.9.2 Входящий журнал 29

2.9.3 Идемпотентность . . . 29
2.10 Архитектура профиля получателя . . . 29
2.11 Архитектура наблюдаемости и контроля качества . . . 29
2.12 Архитектура безопасности . . . 30
2.13 Выводы по разделу . . . 30
3 Реализация системы, моделирование данных и контракты взаимодействия . . . 35
3.1 Технологический стек и модульная структура . . . 35
3.2 Моделирование реляционных данных . . . 36
3.2.1 Схема фасада . . . 36
3.2.2 Схемы канальных сервисов . . . 36
3.2.3 Схемы шаблонов, планировщика и истории . . . 37
3.3 Модель профиля получателя в Redis . . . 37
3.4 Контракты gRPC . . . 38
3.4.1 Контракт фасада . . . 38
3.4.2 Контракт реестра шаблонов . . . 38
3.4.3 Контракты профиля получателя и истории . . . 39
3.5 Реализация рендеринга шаблонов . . . 39
3.6 Реализация фасадного пайплайна . . . 40
3.6.1 Создание события . . . 40
3.6.2 Запуск доставки . . . 40
3.7 Реализация sender-сервисов . . . 41
3.7.1 Почтовый контур . . . 41
3.7.2 SMS-контур . . . 41
3.8 Реализация сервиса истории . . . 42
3.9 Конфигурация и эксплуатационные ограничения текущей реализации . . . 42
3.10 Выводы по разделу . . . 43
4 Развёртывание, эксплуатация и тестирование . . . 44
4.1 Организация развёртывания . . . 44
4.2 Автоматизация выкладки . . . 44
4.3 Средства наблюдаемости . . . 45
4.4 Цели тестирования . . . 45
4.5 Методика проверки . . . 46
4.6 Контрольные критерии успешности . . . 46

4.7 Проверка отказоустойчивости 47
4.8 Нагрузочный прогон как проверка качества пайплайна 47
4.9 Качественный анализ метрик 48
4.10 Интерпретация результатов 49
4.11 Выводы по разделу 49
5 Технико-экономическая оценка и анализ эксплуатационных рисков . 50
5.1 Подход к технико-экономической оценке 50
5.2 Основные статьи затрат 50
5.2.1 Затраты на разработку 50
5.2.2 Инфраструктурные затраты 51
5.2.3 Эксплуатационные затраты 51
5.3 Экономический эффект от архитектурных решений 52
5.3.1 Эффект от разделения жизненного цикла 52
5.3.2 Эффект от outbox/inbox-подхода 52
5.3.3 Эффект от централизованного профиля получателя 52
5.3.4 Эффект от наблюдаемости 52
5.4 Сравнительная оценка вариантов 52
5.5 Анализ рисков информационной безопасности 53
5.6 Влияние архитектуры на стоимость сопровождения 54
5.7 Выводы по разделу 54
ЗАКЛЮЧЕНИЕ 56
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 59
ПРИЛОЖЕНИЕ А Контракты сервисного взаимодействия 62
0.8 Контракт фасада уведомлений 62
0.9 Контракт реестра шаблонов 63
0.10 Контракт профиля получателя 63
0.11 Контракт сервиса истории 63
0.12 Совместимость и эволюция контрактов 64
ПРИЛОЖЕНИЕ Б Kafka-топики и поток сообщений 65
0.13 Основные топики платформы 65
0.14 Формат интеграционного сообщения 65
0.15 Поток EventCreated 66
0.16 Поток TriggerDispatch 66
0.17 Поток статусных событий 66
0.18 Архитектурные свойства Kafka-слоя 67

ПРИЛОЖЕНИЕ В Модель данных и хранилища 68
0.19 Схемы PostgreSQL 68
0.20 Ключевые ограничения и идентификаторы 68
0.21 Служебные журналы 69
0.22 Redis-модель профиля получателя 69
0.23 Связь между уровнями данных 69
ПРИЛОЖЕНИЕ Г Артефакты наблюдаемости и метрик качества 71
ПРИЛОЖЕНИЕ Д Допущения, ограничения и проверка полноты рукописи 75
0.24 Проверка полноты обязательных частей 75
0.25 Критические допущения и ограничения 75
0.26 Проверка целостности текста 75

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Событие — доменный факт, для которого требуется доставка уведомлений целевой аудитории

Запуск доставки — экземпляр исполнения события с фиксированной аудиторией и параметрами отправки

Доставка — запись о доставке по конкретному получателю и каналу в рамках одного запуска доставки

Шаблон — версионируемое описание содержимого уведомления для конкретного канала связи

Канальный шлюз — внешний или внутренний компонент, выполняющий фактическую передачу уведомления по выбранному каналу

Выходящий журнал — паттерн надёжной публикации интеграционных сообщений, при котором бизнес-изменение и запись сообщения фиксируются в одной транзакции базы данных

Входящий журнал — паттерн надёжного приёма, в котором входящее сообщение сначала фиксируется в локальной базе данных, а бизнес-обработка выполняется отдельным шагом

Идемпотентность — свойство операции, при котором повторный запуск с теми же входными параметрами не приводит к дополнительным побочным эффектам

Дедупликация — обнаружение и подавление дублирующихся сообщений или операций по устойчивому ключу

Повторная попытка — контролируемое повторное выполнение операции после временной ошибки

Согласованность в конечном счёте — модель согласованности распределённой системы, в которой синхронизация состояний между сервисами происходит за конечное, но не мгновенное время

at-least-once — модель доставки сообщений, при которой каждое сообщение доставляется получателю не менее одного раза; возможны дубликаты, которые должны обрабатываться на уровне приложения

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

gRPC — удалённый вызов процедур по протоколу HTTP/2

Kafka — распределённая платформа потоковой передачи событий

API — программный интерфейс приложения

CI — непрерывная интеграция

CD — непрерывная доставка

СУБД — система управления базами данных

SLA — соглашение об уровне сервиса

MTTR — среднее время восстановления

JWT — маркер веб-токена

REST — архитектурный стиль представления состояния ресурса

ВВЕДЕНИЕ

Актуальность темы и постановка проблемы

Цифровые продукты практически в любой сфере деятельности опираются на уведомления как на базовый механизм взаимодействия с пользователем. Через них подтверждаются операции, информируются статусы различных заявок заявки, срабатывают напоминания о сроках, инициируются повторные вовлечения пользователя, фиксируются наступления событий, требующих реакции, и многое другое. По мере роста числа систем внутри одной компнаии и по мере роста количества получателей, уведомление перестаёт быть единичным действием и превращается в распределённый процесс, включающий принятие команды, подготовку контента, проверку ограничений получателя, маршрутизацию по каналам связи и непосредественно доставка [1; 2; 3].

Внутри большой корпорации недостаточно выполнить единичный синхронный вызов внешнего провайдера внутри конкретной системы, которая инициирует доставку. В противном случае не избежать большого количества проблем в подходе, когда каждая отдельная система сама обеспечивает доставку тех или иных пользовательских уведомлений. Такое решение приводит к рассогласованности данных по пользовательским предпочтениям, к отсутствию контроля персональных настроек для определенных пользователей, вызывает сложности с анализом всех исходящих пользовательских уведомлений и ко многим другим неприятным последствиям.

В платформе пользовательских уведомлений необходимо гарантировать, что событие, принятое системой, не будет потеряно при частичной недоступности того или иного компонента системы. Кроме того, система должна отличать временный сбой от окончательного отказа, предотвращать повторные и дублирующие доставки, а также сохранять полную прозрачность всей цепочки доставок, позволяющий восстановить путь уведомления от клиентского запроса до финального статуса. Именно сочетание этих требований делает задачу проектирования надёжной платформы уведомлений более сложной, чем реализация доставки по отдельному каналу связи в рамках отдельной системы. [4; 5].

Дополнительную сложность создаёт работа с коммуникационным

профилем получателя. Информация о согласиях, блокировках, предпочтительных каналах связи, наличии контактных данных для доставки, не должна дублироваться в каждом канальном сервисе. При распределённой реализации такая децентрализация неизбежно приводит к рассинхронизации правил, росту количества точек отказа и удорожанию сопровождения. Поэтому в данной работе логика проверки доступности канала и контактных данных выделяется в самостоятельный сервис profile-consent.

Степень разработанности проблемы

Современная инженерная литература и официальная документация по распределённым системам предлагают паттерны надёжной асинхронной интеграции, в том числе: transactional outbox, transactional inbox, идемпотентность на уровне запроса, retry в различной конфигурации, ограничение влияния отдельных доставок друг на друга через локальные транзакции и изоляцию каналов по сервисным границам [4; 2; 5]. Практические аспекты применения этих паттернов подробно раскрыты в документации Apache Kafka, PostgreSQL, Redis, Mongo DB, Spring Framework и gRPC [1; 6; 7; 8; 9; 10; 11].

При этом готовых прикладных решений, которые одновременно решают задачу приёма команд, предварительного рендеринга шаблонов, централизованной проверки профиля получателя, отложенной отправки, публикации в канальные сервисы и накопления истории статусов в рамках одной согласованной модели, немного. И практически все существующие - собственные разработки крупных корпораций под требуемые нужды. На практике такие платформы часто строятся эволюционно, что приводит к накоплению связности, дублированию правил и ухудшению наблюдаемости. Текущая работа ориентирована на проектирование цельной архитектуры и её реализацию на основе существующей микросервисной кодовой базы.

Объект, предмет, цель и задачи исследования

Объект исследования – процессы генерации, маршрутизации, доставки и учёта пользовательских уведомлений в распределённой информационной системе.

Предмет исследования – архитектурные, алгоритмические и

программные механизмы, обеспечивающие устойчивую обработку уведомлений, согласованную передачу данных между сервисами и контролируемую доставку сообщений по нескольким каналам связи.

Цель работы – разработать и реализовать архитектуру отказоустойчивой распределённой платформы пользовательских уведомлений, обеспечивающей гарантию обработки, централизованное определение правил доставки и работа с согласиями для отдельных профилей получателя, поддержку отложенной отправки и полную прозрачность жизненного цикла уведомления.

Для достижения поставленной цели решаются следующие задачи:

а) выполнить анализ предметной области платформ уведомлений, их архитектурных вариантов и типичных отказов;
б) сформулировать функциональные и нефункциональные требования к платформе;
в) обосновать выбор сочетания протоколов и технологий для межсервисного взаимодействия;
г) спроектировать доменную модель события, запуска доставки и отдельной доставки;
д) спроектировать модель данных и границы владения данными между сервисами;
е) реализовать сервис приёма команд notification-facade и сервис предварительного рендеринга шаблонов template-registry;
ж) реализовать централизованный сервис коммуникационного профиля получателя profile-consent;
и) реализовать сервисы для различных каналах связи, а также сервис истории доставок;
к) реализовать контур отложенной публикации через сервис scheduler-delivery;
л) обеспечить реализацию паттернов надежной интеграции;
м) обеспечить наблюдаемость и контроль ключевых метрик качества;
н) провести стендовую проверку основных сценариев и проанализировать результаты нагрузочного прогона.

Методы исследования

В работе применяются следующие методы:

а) системный анализ требований и декомпозиция уведомительного процесса на устойчивые сервисные стадии;
б) сравнительный анализ архитектурных альтернатив: монолитной, модульной и микросервисной событийной архитектуры;
в) сравнительный анализ транспортов и протоколов для межсервисного взаимодействия: REST, gRPC, Kafka и RabbitMQ;
г) проектирование реляционной модели данных средствами SQL-миграций Flyway и использование Redis как оперативного хранилища профилей [12; 13];
д) архитектурное моделирование контейнерных представлений и потоков взаимодействия средствами C4-подхода и Structurizr DSL [14; 15];
е) проверка корректности и устойчивости системы по данным метрик, логов и накопленной истории доставки.

Научная новизна и практическая значимость

Научная новизна работы состоит в том, что уведомительный процесс рассматривается как последовательность явно отделённых состояний и сервисных стадий: принятие доменного события, формирование запуска доставки, обработка конкретной канальной доставки и агрегация истории. Надёжность при этом обеспечивается не единственным механизмом, а композиционно:

а) запись уведомления фиксируются в одной локальной транзакции через transactional outbox;
б) обработчики канальных сообщений принимают Kafka-событие сначала во входящий журнал, а уже затем переходят непосредственно к доставке;
в) retry на всех уровнях остаются безопасными посредством ключей идемпотентности и ограничений на уровне баз данных;
г) проверки согласий пользователя, доступности канала связи и предпочтений получателя вынесены в отдельный сервисный контур;
д) история доставки строится как отдельная проекция событий, пригодная для последующего анализа.

Практическая значимость работы заключается в возможности

использования разработанной платформы как базового контура корпоративных уведомлений. Предложенное решение снижает риск потери сообщений без явной причины, упрощает добавление новых каналов связи посредством гибкой архитектуры, централизует правила маршрутизации и уменьшает время поиска и решения различных инцидентов.

Краткое описание структуры работы

В аналитической главе исследуется предметная область платформ пользовательских уведомлений, формулируются требования и обосновывается выбор архитектурного подхода. В следующей главе проектируется архитектура распределённой платформы и детально описываются потоки данных между сервисами. Далее рассматриваются конкретные части реализации системы, начиная от модели данных, api контрактов, механизмов рендеринга шаблонов до обработки очередей и накопления истории доставок. Отдельная глава посвящена развёртыванию, наблюдаемости и нагрузочному тестированию на локальном стенде. Завершают работу технико-экономическая оценка, анализ эксплуатационных ограничений, заключение, список источников и приложения.

Структура и оформление пояснительной записки выполнены с опорой на ГОСТ 7.32-2017, а библиографические ссылки и описание источников оформлены в соответствии с ГОСТ Р 7.0.5-2008 и ГОСТ Р 7.0.100-2018 [16; 17; 18].

Допущение: В текущий LaTeX-проект не включены отдельные файлы задания на ВКР и календарного плана; согласно шаблону института они оформляются как внешние документы и должны быть приложены к итоговому комплекту работы отдельно.

1 Аналитическая глава, постановка задачи и требования

1.1 Платформа уведомлений как прикладная распределённая система

Платформа уведомлений преобразует доменное событие в набор конкретных действий, обеспечивающих доставку пользовательского уведомления. В простейшей схеме такое действие выглядит как единичный вызов канального шлюза и обработку ответа от этого шлюза. В моей системе процесс доставки уведомления должен включать следующие пункты:

а) валидацию входной команды и проверку идемпотентности;
б) выбор или рендеринг содержимого уведомления, в зависимости от входных данных;
в) определение аудитории и фиксацию списка получателей, для которой необходимо осуществить доставку;
г) проверку доступности канала связи и профилей получателей;
д) фактическую отправку уведомления;
е) фиксацию статуса доставки и сохранение в истории.

Следовательно, уведомление в распределённом контуре нельзя свести к одному RPC вызову. Оно представляет собой составной бизнес-процесс, разбитый на несколько локально транзакционных стадий, между которыми возможны задержки, дублирующие публикации, повторные попытки и частичные отказы.

1.2 Проблемы прямой синхронной схемы доставки

Прямой подход, при котором прикладная система синхронно вызывает канальный шлюз привлекателен простотой реализации, но подобное решение плохо масштабируется по мере роста интеграционных каналов связи, числа пользователей и общего числа различных систем в рамках одной компании или одного продукта. Для предметной области доставки уведомлений он порождает сразу несколько ключевых проблем:

а) отказ канального шлюза непосредственно влияет на время ответа на клиентский запрос;
б) повторная отправка запроса на верхнем уровне может привести к дублям доставки;

в) сложно различать различные стадии доставки уведомлений;
г) невозможно независимо масштабировать планирование доставки и каналы связи;
д) правила согласий, блокировок и предпочтений учитываются сразу в нескольких местах для каждого канала отдельно.

Для платформы уведомлений особенно критично последнее обстоятельство. Если каждая система-отправитель самостоятельно обращается к профилю пользователя и самостоятельно трактует признаки согласия или доступности канала, то уже через несколько интеграций эта система утрачивает единый источник истины или пораждается сильная связанность двух систем. В результате одна и та же коммуникационная политика начинает интерпретироваться по-разному в разных частях платформы, что неминуемо будет приводить к неожиданному поведению для конечного пользователя.

1.3 Доменная модель уведомительного процесса

Для решения указанной проблемы в работе используется трёхуровневая доменная модель.

Событие уведомления фиксирует факт того, что должно быть подготовлено уведомление с определённым шаблоном, приоритетом, полезной нагрузкой, аудиторией и предпочтительным каналом связи. Событие имеет возможность настройки, изменения и переиспользования. Оно описывает предметный контекст будущей отправки.

Запуск доставки представляет собой отдельный поток исполнения события. Он содержит snapshot аудитории и параметров маршрутизации на момент запуска во избежание несогласованности данных ввиду возможности их изменения во время процесса доставки. Разделение события и запуска доставки уведомлений позволяет гибко управлять временем и контекстом процесса.

Канальная доставка отражает результат доставки после всех проверок для конкретной пары «получатель – канал». Именно на этом уровне фиксируются вся мета-информация и фактическая история доставок.

Такое разделение упрощает анализ и управление платформой, потому что событие описывает намерение, запуск доставки описывает триггер к началу обработки, а доставка описывает независмисый итог для конкретного

адресата на конкретный момент времени.

1.4 Архитектурные варианты и обоснование выбора

Для предметной области уведомлений были рассмотрены четыре варианта архитектуры:

а) монолитный сервис со встроенными адаптерами под каждый канал связи;
б) модульный монолит с единым хранилищем и внутренними пакетными обработчиками доставок;
в) набор микросервисов с полностью синхронными взаимодействиями внутри системы;
г) микросервисная архитектура с разделением командного и фонового контуров с элементами событиного подхода.

Монолитная архитеутра упрощает старт и уменьшает операционные издержки. Так же монолит плохо изолирует отказы каналов и не позволяет независимо масштабировать разные стадии жизненного цикла обработки запроса. Модульный монолит делает кодовую базу более понятной, однако всё равно сохраняет общую релизную границу и не обеспечивает независимость компонентов, которая необходима для платформы уведомлений.

Полностью синхронная микросервисная схема решает проблему размерности кодовой базы, но не устраняет главную трудность. Длительные и потенциально уязвимые к ошибкам этапы доставки всё равно остаются привязанными к critical path обработки входного запроса, что не позволяет обрабатывать данные инциденты обособленно. Поэтому в работе выбран четвёртый вариант, а именно: контур gRPC используется для приёма команд и чтения состояния. Непосредственно публикация, отложенная отправка и канальная доставка выполняются через взаимодействие сервисов посредством технологии Apache Kafka [11; 1; 3].

1.5 Сравнительный анализ транспортов межсервисного взаимодействия

Сопоставление ключевых технологических вариантов приведено в таблице 1.

Выбор Kafka обусловлен тем, что платформа должна не только

доставлять команды к обработчику, но и хранить поток интеграционных событий, необходимый для случаев отложенной обработки, повторной публикации, независимого потребления несколькими сервисами и так же для случаев восстановления во время сбоев в работе [1; 6; 19].

1.6 Варианты работы с профилем получателя

Отдельного анализа требует выбор способа доступа к профилю получателя. Возможные варианты приведены в таблице 2.

Использование Redis как оперативного хранилища профилей оправдано тем, что проверка доступности канала относится к коротким и часто повторяющимся операциям, а сами профили естественно представимы в виде компактных hash-структур [13]. При этом владельцем модели данных остаётся сервис profile-consent, а не каждый канальный sender. Наполняемость данных реализовывается различными способами. Выбор в пользу данного NoSQL хранилища был сделан в том числе с использованием допущения о том что платформа пользовательских уведомлений не должна быть источником истины для пользовательских данных, поэтому будем считать далее, что ответстенность за польовательские данные лежит целиком и полностью на обособленной системе.

1.7 Постановка задачи

Исходя из анализа предметной области, ставится задача разработать платформу, которая принимает команды на создание событий, хранит их состояние, при необходимости выполняет предварительный рендеринг шаблонов, формирует запуск доставки, публикует его в событийный контур, выполняет канальную обработку, проверяет профиль согласия и предочтения получателя и накапливает историю статусов по каждому получателю и каналу в отдельном сервисе.

Ключевой особенностью постановки задачи является отказ от прямой зависимости между клиентским запросом и фактической отправкой уведомления. Клиент должен получить подтверждение о принятии команды и идентификатор созданного объекта. Дальнейшая обработка должна выполняться асинхронно, но с возможностью проследить каждую стадию и повторно запустить обработку без потери согласованности данных, если это

возможно.

1.7.1 Функциональные требования

На основании поставленной задачи формируются следующие функциональные требования:

а) создание, изменение, отмена, чтение и постраничный просмотр доменных событий;
б) управление аудиторией события, включая явный список получателей;
в) запуск доставки по выбранному событию с поддержкой немедленного или отложенного времени публикации;
г) хранение и рендеринг версионируемых шаблонов уведомлений;
д) централизованная проверка профиля получателя: активности профиля, доступности канала, чёрных списков и адреса доставки;
е) приём запусков непосредственной канальной доставки через Kafka;
ж) фиксация статусов и истории попыток отправки;
и) возможность предоставление агрегированной истории доставки и сводок по получателю;
к) поддержка расширения на дополнительные каналы без изменения базовой доменной модели.

1.7.2 Нефункциональные требования

Нефункциональные требования приведены в таблице 3.

1.7.3 Требования к надёжности и обработке ошибок

Платформа должна удовлетворять следующим требованиям надёжности:

а) публикация событий в Kafka должна выполняться через выходящий журнал, записываемый в той же транзакции, что и бизнес-состояние;
б) канальный сервис не должен выполнять бизнес-обработку непосредственно из метода Kafka listener без предварительной фиксации сообщения во входящем журнале;
в) retries на всех уровнях взаиодействия должны быть явно ограничены и диагностируемы;
г) временные ошибки и окончательные ошибки должны различаться по

состояниям и истории;

д) потеря экземпляра отедльного сервиса не должна требовать ручного восстановления всей очереди обработки.

1.7.4 Требования к данным и хранению

Для данных и хранилищ формируются следующие требования:

а) доменные данные и технические журналы должны храниться в персистентных базах данных;

б) миграции схем должны быть воспроизводимы и управляться централизованно [12];

в) система должна использовать отдельные логические схемы данных для фасада, планировщика, канальных обработчиков, истории и шаблонов;

г) полезная нагрузка уведомления в jsonb должна храниться как технический контекст обработки, а не как единственный источник бизнес-истины.

1.8 Типовые сценарии использования

Сценарий 1. Создание события. Клиент вызывает метод CreateNotificationEvent, передавая шаблон, полезную нагрузку, приоритет и аудиторию. Платформа валидирует запрос, подавляет дубли по idempotency key, при необходимости рендерит шаблон и фиксирует событие.

Сценарий 2. Ручной запуск доставки. Клиент вызывает TriggerDispatch. Платформа создаёт запись запуска доставки, снимок аудитории и публикует канальное сообщение либо сразу в рабочий топик, либо в топик отложенной публикации.

Сценарий 3. Отложенная отправка. Если момент публикации находится в будущем, сообщение попадает в сервис scheduler-delivery, который хранит задачу до наступления времени и затем снова публикует её в необходимый канал.

Сценарий 4. Канальная обработка. Email- или SMS-sender принимает сообщение из Kafka, сохраняет его во входящий журнал, проверяет профиль получателя, создаёт либо отложенную доставку, либо явный пропуск с указанием причины, после чего фактический worker выполняет отправку.

Сценарий 5. История и отчётность. После изменения статуса канальной доставки соответствующий sender публикует событие в Kafka, отражающее статус доставки, а history-writer сохраняет его в нормализованную историю и предоставляет агрегированные запросы по получателю.

1.9 Выводы по разделу

Проведённый анализ показывает, что для платформы уведомлений оправдана архитектура, в которой синхронный и асинхронный контуры разделены. Процесс обработки входящего клиентского запроса на доставку явно декомпозирован на событие, запуск доставки и отдельную канальную отправку. Наиболее рациональным вариантом для рассматриваемой предметной области является сочетание gRPC как строгого контрактного интерфейса и Kafka как событийной шины взаимодействия внутренних сервисов. Отдельным принципиальным решением является выделение сервиса профиля получателя, который устраняет дублирование правил в канальных обработчиках и повышает согласованность платформы в целом [3; 4; 1; 11; 13].

Таблица 1 – Сравнительный анализ транспортов межсервисного взаимодействия

Технология	**Преимущества**	**Ограничения**	**Роль в системе**
gRPC	Строгая схема с protobuf контрактами, низкие накладные расходы, гибкая эволюция контракта	Неприменим для длительных фоновых процессов и неудобен как транспорт массовой fan-out публикации	Контракты для взаимодействия с внешними апи, общение внутри системы с сервисами template-registry, profile-consent и history-writer
REST	Простота интеграции и развитая экосистема HTTP-инструментов, проще точка входа для имплементации	Более слабая типизация, отсутствие строгого контракта между клиентом и сервером, большие накладные расходы на уровне протоколов	Рассматривался как внешний интерфейс, но не выбран как основной внутренний протокол
Kafka	Высокая пропускная способность, устойчивость к всплескам нагрузки, гибкое горизонтальное масштабирование	Требует прикладной идемпотентности, контроля дубликатов и детального проектирования событий	Основная событийная шина для публикации событий входящих запросов
RabbitMQ	Удобная модель очередей и подтверждений, богатая маршрутизация по exchange/queue сущностям	Менее естественная модель для долговременных потоков событий и повторного чтения журналов по сравнению с Kafka, менее гибкий брокер к горизонтальному масштабированию	Рассматривался как альтернатива для обработки входящих в систему событий, но не выбран как основа событийного контура

Таблица 2 – Сравнение вариантов доступа к коммуникационному профилю получателя

Вариант	**Преимущества**	**Недостатки**	**Вывод**
Прямой доступ каждого sender к общей базе или кэшу	Минимум промежуточных сервисов	Дублирование логики, сильная связанность, неконтролируемое распространение правил и доступов	Не выбран
Локальная копия профиля внутри каждого sender	Низкая latency принятия решения	Сложная синхронизация, риск неконсистентных данных, дорогая поддержка нескольких моделей	Не выбран
Выделенный сервис коммуникационного профиля пользователя	Единая бизнес-логика, единый контракт на всех обработчиков, упрощённое расширение модели канала	Дополнительный hop при вызове sender, необходимость поддерживать отдельный сервис	Выбран

Таблица 3 – Нефункциональные требования платформы

Категория	**Требование**
Надёжность	Уже принятое событие не должно бесследно исчезать при сбое на любом из этапов обработки; восстановление обработки должно быть возможно из локального персистентного состояния.
Идемпотентность	Повторные клиентские вызовы с одинаковым содержимым не должны приводить к созданию дубликатов на любом уровне.
Масштабируемость	Приём команд, отложенная публикация событий, канальные обработчики и сервис истории должны масштабироваться независимо друг от друга.
Наблюдаемость	Должны собираться метрики gRPC и Kafka-взаимодействий, статусы доставок и технические логи, достаточные для анализа цепочки обработки.
Гибкость сопровождения	Добавление нового канала связи не должно требовать значительной переработки любых сервисов в рамках системы.

2 Проектирование архитектуры распределённой платформы

2.1 Архитектурные принципы

Архитектура разрабатываемой платформы построена как набор микросервисов с явным разделением доменных обязанностей и собственных контуров хранения. Такой подход выбран в соответствии с требованиями к разделению потока обработки и независимому масштабированию отдельных компонентов. Система должна одновременно принимать клиентские команды, хранить жизненный цикл событий обработки, иметь возможность выполнять массовую массовую канальную доставку, учитывать профиль и ограничения получателя, фиксировать историю, выдерживать повторные вызовы и восстанавливаться после сбоев без глобальной распределённой транзакции [3; 4; 2].

В качестве базовых архитектурных принципов приняты:

а) разделение синхронного командного контура и асинхронного контура массовой обработки;

б) локальная транзакционность внутри каждого сервиса при отказе от подходов к распределенным транзакциям;

в) обязательная идемпотентность входных команд и межсервисных взаимодействий;

г) сохранение промежуточных состояний обработки;

д) централизованное разрешение коммуникационного профиля получателя;

е) наблюдаемость каждого этапа через метрики, логи и явные доменные статусы.

Следствием этих принципов является комбинированная схема взаимодействия. gRPC используется там, где вызывающей стороне нужен непосредственный ответ и жёсткая валидация структуры запроса. Kafka используется для всех остальных этапов обработки, где важнее не мгновенный ответ, а надёжная передача состояния между сервисами с возможностью накапливать небольшую задержку на обработку [11; 1; 5].

Внешние границы системы и её интеграционные связи показаны на рисунке 1.

2.2 Контейнерная декомпозиция

Актуальная контейнерная модель платформы формализована также в Structurizr DSL. [15; 14].

Контейнерное представление текущей реализации приведено на рисунке 2. Диаграмма фиксирует фактический состав сервисов, хранилищ и наблюдаемости, а также основные каналы обмена между ними.

В текущей реализации система включает сервисы, приведённые в таблице 4.

Такое разбиение уменьшает связанность и локализует отказ. Например, временная недоступность SMTP-шлюза не блокирует приём новых команд на фасаде, а проблема в сервисе истории не отменяет уже созданные доставки, поскольку история обновляется асинхронно. Основная идея подхода в том чтобы добиться максимально низкой связанности сервисов для возможности гибкого масштабирования.

2.3 Границы ответственности и модель данных верхнего уровня

Верхнеуровневая доменная модель включает три различных уровня сущностей:

а) **событие уведомления** — бизнес-объект, описывающий намерение выполнить рассылку;

б) **запуск доставки** — процесс исполнения события с зафиксированной аудиторией и временем отправки;

в) **канальная доставка** — результат обработки по конкретному получателю и каналу связи.

Такое разделение принципиально важно. Событие отражает бизнес-замысел клиента, запуск доставки фиксирует момент начала обработки и выбранную аудиторию, а канальная доставка содержит операционный результат по отдельному адресату. Если смешивать эти уровни, становится трудно поддерживать повторный запуск, отложенную отправку, частичную неуспешность и историю по получателю.

2.4 Поток данных при создании события

Поток обработки команды CreateNotificationEvent организован следующим образом.

а) Клиентская система вызывает gRPC-метод фасада и передаёт идентификатор шаблона, версию, предпочитаемый канал, стратегию отправки, ключ идемпотентности и полезную нагрузку.
б) Фасад проверяет наличие события с той же парой client_id и idempotency_key. При повторе команда не создаёт новое событие, а возвращает уже существующее состояние.
в) Фасад формирует полезную нагрузку доставки. Если в запросе уже присутствуют поля subject и body либо их эквиваленты, встроенный текст используется без вызова реестра шаблонов. Если встроенного текста нет и заданы template_id и канал, фасад вызывает RenderPreview сервиса Template Registry и получает отрендеренные subject/body.
г) В локальной транзакции сохраняются запись события, описание аудитории и, при явной аудитории, список получателей.
д) В той же транзакции создаётся запись выходящего журнала типа notification_event с событием EventCreated.
е) Отдельный фоновый relay-процесс публикует запись в Kafka-топик notification.facade.events.

Сценарий создания события и публикации EventCreated в динамике представлен на рисунке 3. Диаграмма визуализирует ключевые переходы между фасадом, реестром шаблонов, хранилищем и Kafka.

Ключевое свойство этого сценария состоит в том, что клиент получает подтверждение только после фиксации события и соответствующей outbox-записи. Публикация в Kafka выполняется отдельно, но уже из устойчиво сохранённого состояния, что соответствует паттерну transactional outbox [4; 2].

2.5 Поток данных при запуске доставки

Команда TriggerDispatch переводит подготовленное событие в стадию фактической доставки. В текущей реализации обработка выполняется так:

а) фасад находит событие и проверяет, что оно не отменено;
б) из таблицы аудитории извлекается описание получателей;
в) создаётся запись dispatch с собственным ключом идемпотентности и вычисленным временем отправки;
г) если аудитория задана как EXPLICIT и включён режим

snapshotOnDispatch, список получателей копируется в dispatch_target;

д) в выходящий журнал фасада записывается либо MailDispatchRequested, либо SmsDispatchRequested в зависимости от preferred_channel;

е) outbox relay фасада маршрутизирует запись в соответствующий Kafka-топик.

Для почтового канала дополнительно реализована логика отложенной публикации. Если поле planned_send_at указывает на будущий момент, запись типа mail_dispatch маршрутизируется не в рабочий топик notification.mail.dispatches, а в notification.mail.dispatches.scheduled. Это позволяет отделить логику планирования времени отправки от логики исполнения канальной доставки.

2.6 Поток данных от планировщика к канальному сервису

Сервис scheduler-delivery реализует отдельный асинхронный слой для отложенной почтовой доставки:

а) консьюмер читает сообщения из notification.mail.dispatches.scheduled;

б) входящее сообщение нормализуется и сохраняется в nf_sched.scheduled_delivery_task;

в) периодический publisher выбирает только те задачи, у которых наступило время исполнения;

г) при успешной публикации сообщение повторно отправляется в рабочий топик notification.mail.dispatches, после чего задача помечается как опубликованная;

д) при ошибке задача не теряется, а переводится в повторную попытку с новой датой retry_at.

Таким образом, отложенная отправка не реализуется через сон потока, cron внутри фасада или хранение таймеров в памяти. Она оформлена как отдельный устойчивый подконтур, который можно масштабировать и перезапускать независимо от фасада.

2.7 Поток данных в канальных сервисах

Почтовый и SMS-каналы реализованы как два отдельных сервиса, но используют общую архитектурную схему.

2.7.1 Этап транспортного приёма

Kafka-консьюмер не выполняет всю бизнес-логику непосредственно в методе @KafkaListener. Сначала сообщение десериализуется и сохраняется во входящий журнал consumer_inbox_message. Лишь затем фоновый workflow забирает записи из этой таблицы и создаёт доменные записи доставки. Такое разделение защищает от потери сообщения при сбое между чтением из Kafka и записью в прикладные таблицы.

2.7.2 Этап планирования доставки

В каждом sender-сервисе имеются два независимых сценария планирования:

а) обработка события EventCreated из notification.facade.events;
б) обработка канального запуска MailDispatchRequested или SmsDispatchRequested.

Оба сценария используют сервис оценки допустимости доставки. Для email это MailDeliveryPlanService, для SMS — SmsDeliveryPlanService. На этапе планирования выполняются:

а) обращение к profile-consent для проверки согласия и получения адреса канала;
б) проверка истории доставки в заданном окне времени;
в) формирование записи PENDING либо SKIPPED с явной причиной отказа.

2.7.3 Этап фактической отправки

Из таблиц mail_delivery и sms_delivery периодически выбираются записи в состояниях ожидания или повторной попытки. Перед отправкой создаётся запись о попытке, а сама доставка переводится в состояние исполнения. После ответа шлюза сервис либо фиксирует успех, либо рассчитывает следующую повторную попытку, либо закрывает доставку как

окончательно неуспешную.

Отдельно важно отметить различие каналов. Почтовый сервис выполняет профильную проверку при планировании доставки, а шлюз SpringMailGateway затем использует уже подготовленные данные. SMS-сервис, помимо проверки на этапе планирования, выполняет дополнительную валидацию batch-пакета в SmsProviderGuardService непосредственно перед отправкой. Это архитектурно допустимо, но создаёт более дорогой по числу вызовов путь обработки.

2.8 Архитектура обмена через Kafka

Основные топики обмена приведены в таблице 5.

Формат интеграционного сообщения унифицирован вокруг outbox-конверта с полями outboxId, aggregateType, aggregateId, eventType, payload, headers и createdAt. Унификация конверта упрощает входящую обработку, трассировку и расширение числа событий без переписывания транспортного слоя.

2.9 Границы транзакций и механизмы надёжности

Архитектура сознательно не использует распределённые транзакции между сервисами. Согласованность достигается через сочетание локальных транзакций, журналов и идемпотентности.

2.9.1 Выходящий журнал

Во фасаде и sender-сервисах доменное изменение и запись в outbox фиксируются в одной транзакции базы данных. Далее relay-процесс:

а) выбирает пакет непубликованных строк;
б) помечает их токеном аренды;
в) публикует в Kafka;
г) подтверждает публикацию только после успешной отправки;
д) освобождает аренду и назначает повторную попытку при ошибке.

Защита от гонок при конкурентной обработке строится на SQL-блокировках FOR UPDATE SKIP LOCKED, что позволяет безопасно распараллеливать выборку задач без двойной обработки [9; 8].

2.9.2 Входящий журнал

В почтовом и SMS-сервисах входящее сообщение сначала записывается во входящий журнал. Лишь после этого отдельный workflow переводит его в прикладные записи доставки. Такая схема соответствует модели at-least-once: повторное чтение сообщения допускается, но компенсируется уникальными ограничениями и идемпотентной обработкой [6; 4].

2.9.3 Идемпотентность

Идемпотентность обеспечивается на нескольких уровнях:

а) уникальность события по паре client_id и idempotency_key;
б) уникальность запуска доставки по паре event_id и idempotency_key;
в) уникальность входящих записей журналов и канальных доставок по ключевым бизнес-идентификаторам;
г) повторное чтение уже созданной сущности при конфликте вставки.

2.10 Архитектура профиля получателя

Сервис profile-consent является единственной точкой входа к данным о получателе, хранящимся в Redis. Остальные сервисы не должны работать с Redis напрямую. Это решение изолирует модель хранения и позволяет развивать формат профиля без изменения логики всех отправителей.

В профиле хранятся:

а) признак активности получателя;
б) предпочитаемый канал;
в) разрешение канала, признак блокировки и destination для email;
г) разрешение канала, признак блокировки и destination для SMS;
д) разрешение канала, признак блокировки и destination для push;
е) время последнего обновления.

Важно подчеркнуть, что наличие push в модели профиля означает готовность контракта к расширению, но не наличие полноценного sender-сервиса push-доставки в текущей кодовой базе.

2.11 Архитектура наблюдаемости и контроля качества

Система проектируется как наблюдаемая с самого начала. Для этого

каждый сервис экспортирует метрики Actuator и Micrometer, а Kafka-клиенты публикуют собственные серии по producer- и listener-операциям. Prometheus выполняет сбор этих метрик, Grafana используется для анализа RPS, задержек, ошибок и утилизации ресурсов [20; 21; 22; 23].

Наблюдаемость в данной архитектуре выполняет две функции:

а) эксплуатационную — позволяет обнаруживать отказ и локализовать узкое место;

б) верификационную — подтверждает прохождение сообщения по всем стадиям от приёма команды до записи истории.

Именно поэтому в работе метрики используются прежде всего как артефакты качества и согласованности потоков данных, а не как единственный критерий ценности системы.

2.12 Архитектура безопасности

С точки зрения проектной модели система должна поддерживать аутентификацию сервисных вызовов, разграничение доступа по роли клиента, шифрование транспортного уровня и безопасное обращение с секретами каналов доставки [24; 25; 26].

Допущение: В демонстрационном профиле текущей реализации часть gRPC-сервисов работает с отключённой аутентификацией, Kafka использует PLAINTEXT, а отдельные gRPC-клиенты создаются через usePlaintext(). Для промышленного контура требуется включение сервисной аутентификации, TLS, централизованного хранения секретов и аудита межсервисных вызовов.

2.13 Выводы по разделу

Спроектированная архитектура задаёт чёткие границы между приёмом команды, планированием отправки, канальной обработкой и историей. Система опирается на gRPC как на синхронный контур управления и на Kafka как на транспорт массовой асинхронной обработки. Надёжность обеспечивается не распределённой транзакцией, а совокупностью локальных транзакций, outbox/inbox-паттернов, идемпотентности, повторных попыток и наблюдаемости. Такой подход соответствует как фактической реализации проекта, так и требованиям к отказоустойчивой платформе уведомлений.

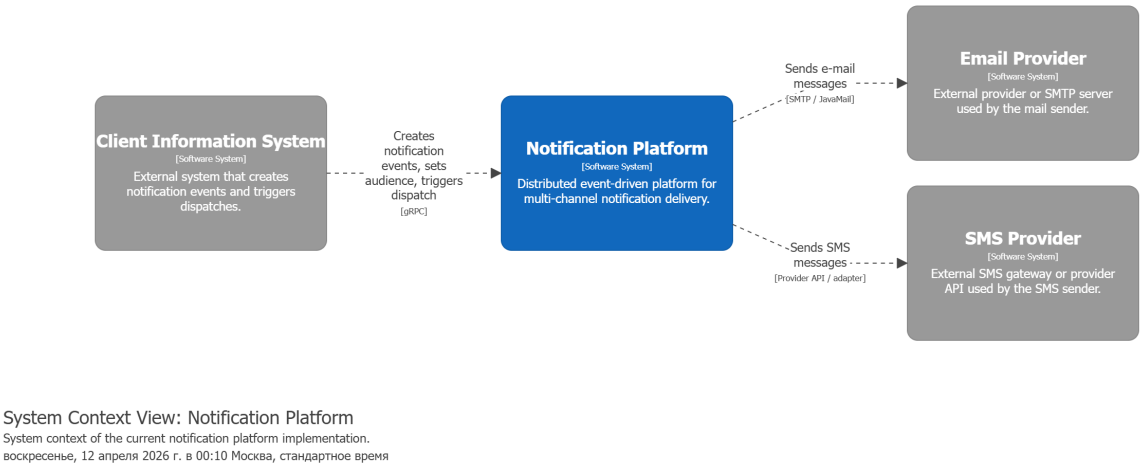


Рисунок 1 – Системный контекст платформы уведомлений (Structurizr System Context)

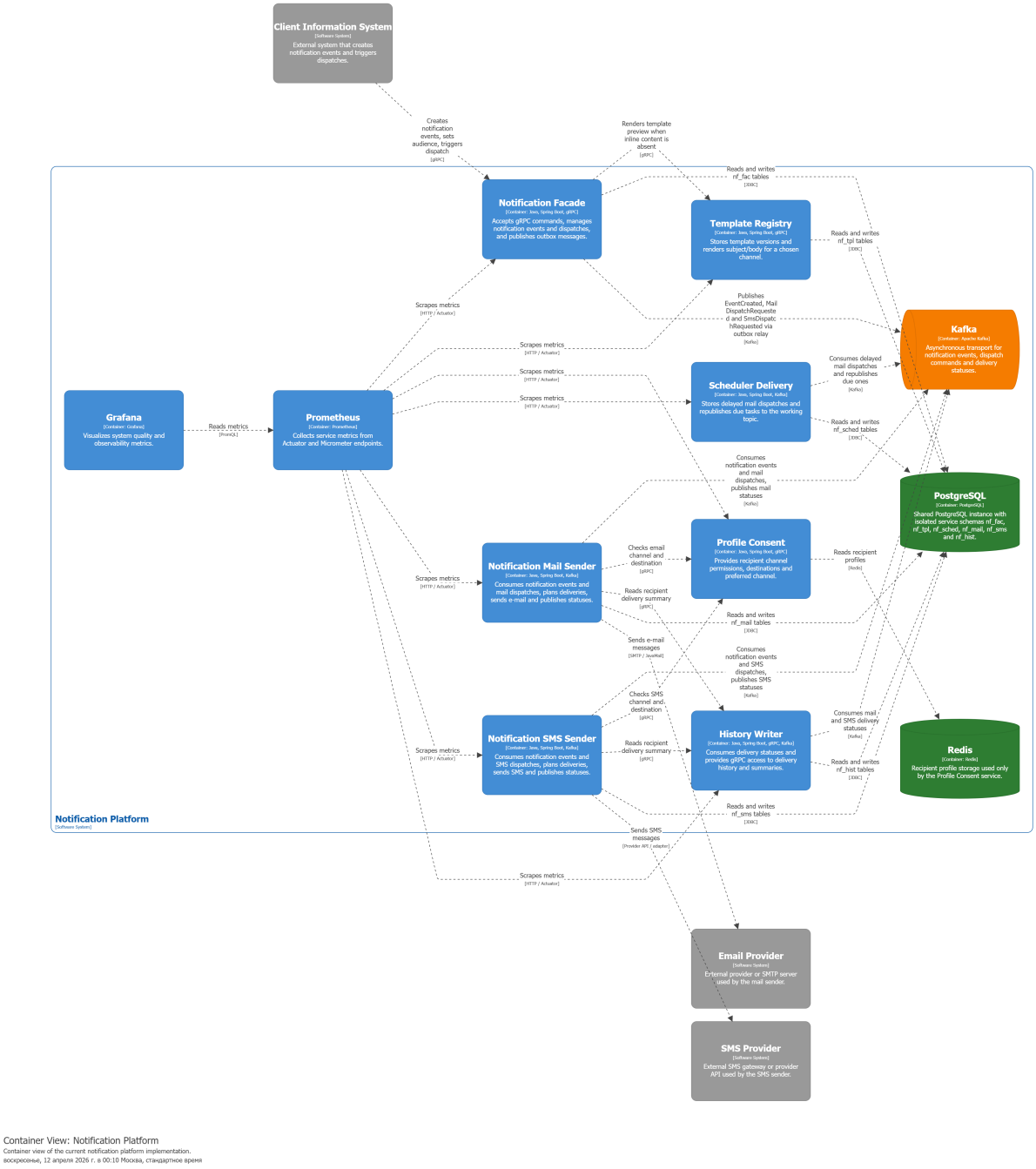
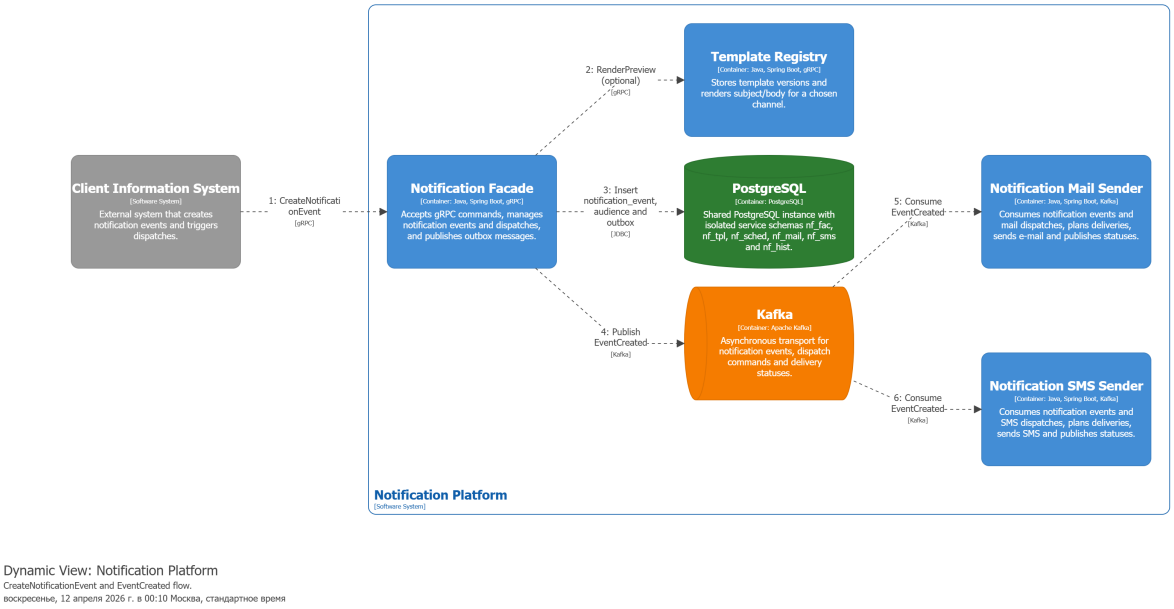


Рисунок 2 – Контейнерная диаграмма платформы уведомлений (Structurizr Container View)

Таблица 4 – Контейнеры и их ответственность

Контейнер	**Назначение**	**Основные зависимости**
Notification Facade	Приём gRPC-команд, валидация, управление событиями и аудиторией, создание запусков доставки, запись в выходящий журнал.	PostgreSQL, Template Registry, Kafka
Template Registry	Хранение шаблонов и версий, выбор контента по каналу, рендеринг subject/body.	PostgreSQL
Profile Consent	Выдача профиля получателя, признаков согласия, блокировок, предпочитаемого канала и адресов доставки.	Redis, gRPC
Scheduler Delivery	Приём отложенных почтовых запусков, хранение задач по времени, повторная публикация в рабочий топик.	PostgreSQL, Kafka
Notification Mail Sender	Приём событий и почтовых запусков из Kafka, планирование и выполнение email-доставок, публикация статусных событий.	PostgreSQL, Kafka, Profile Consent, History, SMTP/JavaMail
Notification SMS Sender	Приём событий и SMS-запусков из Kafka, планирование и выполнение SMS-доставок, публикация статусных событий.	PostgreSQL, Kafka, Profile Consent, History
History Writer	Приём статусных сообщений о доставке и выдача истории по gRPC.	PostgreSQL, Kafka
Kafka	Межсервисная асинхронная передача событий, запусков и статусных сообщений.	Продюсеры и консьюмеры сервисов
PostgreSQL	Персистентное хранение доменного состояния и технических журналов в отдельных схемах сервисов.	Facade, Template Registry, Scheduler, Mail, SMS, History
Redis	Оперативное хранилище профилей получателей.	Profile Consent
Prometheus + Grafana	Сбор и визуализация эксплуатационных метрик сервисов и Kafka-клиентов.	Actuator/Micrometer endpoints



Dynamic View: Notification Platform
CreateNotificationEvent and EventCreated flow.
воскресенье, 12 апреля 2026 г. в 00:10 Москва, стандартное время

Рисунок 3 – Динамический поток CreateNotificationEvent/EventCreated (Structurizr Dynamic View)

Таблица 5 – Ключевые Kafka-топики архитектуры

Топик	**Продюсер**	**Назначение**
notification.facade.events	Notification Facade	События жизненного цикла уведомления, включая EventCreated.
notification.facade.dispatches	Notification Facade	События по сущности dispatch; в текущем коде публикуются фасадом, но выделенного активного потребителя для бизнес-обработки не обнаружено.
notification.mail.dispatches	Notification Facade, Scheduler Delivery	Рабочие команды на почтовую доставку.
notification.mail.dispatches.scheduled	Notification Facade	Отложенные почтовые команды до момента выпуска планировщиком.
notification.sms.dispatches	Notification Facade	Рабочие команды на SMS-доставку.
notification.mail.delivery-statuses	Notification Mail Sender	Статусы и результаты почтовой доставки.
notification.sms.delivery-statuses	Notification SMS Sender	Статусы и результаты SMS-доставки.

3 Реализация системы, моделирование данных и контракты взаимодействия

3.1 Технологический стек и модульная структура

Реализация выполнена как много-модульный Gradle-проект на Java 21 и Spring Boot 3.4.x. Такой формат позволил разделить прикладные сервисы, protobuf-контракты и вспомогательные библиотеки, сохранив единый процесс сборки и единые правила версионирования.

В прикладной части проекта реализованы следующие сервисные модули:

а) notification-facade;
б) template-registry;
в) profile-consent;
г) scheduler-delivery;
д) notification-mail-sender;
е) notification-sms-sender;
ж) history-writer;
и) custom-loader как утилита генерации нагрузки и сервисных сценариев.

Отдельными модулями вынесены protobuf-контракты фасада, профиля получателя, истории и шаблонов. Это позволяет переиспользовать единые .proto-описания в серверной и клиентской частях без дублирования сигнатур.

С точки зрения прикладных библиотек используются:

а) Spring gRPC для серверных и клиентских вызовов;
б) Spring Kafka для transport-слоя и фоновых консьюмеров;
в) Spring JDBC для доступа к PostgreSQL;
г) Flyway для миграций схем данных;
д) Spring Data Redis для чтения профилей коммуникаций;
е) JavaMailSender для почтового адаптера;
ж) Micrometer и Actuator для метрик и диагностических endpoint-ов [11; 27; 10; 12; 13; 21].

Использование Spring JDBC вместо ORM является сознательным решением. Для данной предметной области критичны контролируемые транзакционные границы, прозрачные SQL-запросы, явные индексы и предсказуемое поведение при пакетной обработке очередей и журналов.

Поэтому взаимодействие с реляционной базой данных оформлено через миграции и hand-written SQL, а не через автоматическую генерацию запросов.

3.2 Моделирование реляционных данных

3.2.1 Схема фасада

Фасад использует схему nf_fac, в которой сосредоточены как доменные сущности события уведомления, так и технические записи интеграции. Основные таблицы:

а) notification_event — корневая сущность уведомительного события;
б) event_audience — описание типа аудитории и её параметров;
в) event_recipient — явный список получателей для EXPLICIT-аудитории;
г) dispatch — экземпляр запуска доставки;
д) dispatch_target — снимок получателей конкретного запуска;
е) outbox_message — выходящий журнал интеграционных сообщений.

В таблице notification_event хранятся идентификатор клиента, ключ идемпотентности, идентификатор шаблона, версия, приоритет, предпочитаемый канал, стратегия отправки, статус и JSON-представление полезной нагрузки. На уровне БД задана уникальность по паре client_id + idempotency_key, что обеспечивает защиту от повторного создания одного и того же события.

Таблица dispatch логически отделяет бизнес-событие от факта его запуска в канал. Это особенно важно для отложенной отправки и повторных запусков по одной и той же сущности события. Уникальность event_id + idempotency_key не допускает дублей запусков при сетевых повторах или повторном вызове клиента.

3.2.2 Схемы канальных сервисов

Почтовый сервис использует схему nf_mail, SMS-сервис — nf_sms. Несмотря на различие каналов, структура их хранения одинакова по роли таблиц:

а) consumer_inbox_message — входящий журнал Kafka-сообщений;
б) mail_delivery или sms_delivery — таблица фактических доставок;
в) mail_delivery_attempt или sms_delivery_attempt — история попыток;

г) outbox_message — статусные сообщения для сервиса истории.

В почтовом сервисе дополнительно присутствует таблица recipient_mail_settings, используемая как служебный слой хранения настроек отправки. Канальные таблицы содержат ключи dispatch_id, event_id, client_id, recipient_id, тип канала, статус, payload, идентификатор шаблона, версию шаблона, счётчик попыток, следующий момент отправки и последнюю ошибку.

Такая модель даёт несколько важных эксплуатационных свойств:

а) можно безопасно возобновить обработку после перезапуска сервиса;
б) каждая повторная попытка фиксируется отдельно;
в) состояние канального контура можно анализировать независимо от фасада;
г) история по получателю и каналу строится из фактических, а не вычисляемых состояний.

3.2.3 Схемы шаблонов, планировщика и истории

Реестр шаблонов использует схему nf_tpl с таблицами template и template_version. Таблица template хранит идентичность шаблона и номер активной версии, а template_version — движок рендеринга, JSON-массив канальных представлений и время создания версии.

Планировщик использует схему nf_sched и таблицу scheduled_delivery_task, в которой хранятся входящие отложенные сообщения, тип агрегата, идентификатор, payload, заголовки, время исходного создания, состояние задачи и следующая повторная попытка публикации.

Сервис истории использует схему nf_hist с таблицей delivery_history. В ней сохраняются событие, запуск, получатель, канал, статус, причина, номер попытки, времена возникновения и служебный outbox_id, по которому применяется идемпотентная вставка on conflict do nothing.

3.3 Модель профиля получателя в Redis

Сервис profile-consent не хранит профиль в реляционной модели. Он использует Redis-хеш по ключу с префиксом profile-consent:recipient: и читает из него следующие атрибуты:

а) active;

б) preferred_channel;
в) email.enabled, email.blacklisted, email.destination;
г) sms.enabled, sms.blacklisted, sms.destination;
д) push.enabled, push.blacklisted, push.destination;
е) updated_at.

На выходе эти поля собираются в объект RecipientProfile, содержащий карту Channel -> ChannelConsent. Важно, что Redis-модель остаётся внутренней деталью profile-consent; остальные сервисы работают только с gRPC-ответами и не зависят от конкретных имён полей, ключей и формата хранения.

3.4 Контракты gRPC

3.4.1 Контракт фасада

Сервис NotificationFacade реализует три группы операций:

а) управление событием: CreateNotificationEvent, UpdateNotificationEvent, CancelNotificationEvent, GetNotificationEvent, ListNotificationEvents;
б) управление аудиторией: SetAudience, AddRecipients, RemoveRecipients, GetAudience;
в) управление запуском доставки: TriggerDispatch, GetDispatch, ListDispatches.

Такое разбиение отделяет создание и редактирование события от факта запуска массовой обработки. С точки зрения проектирования API это существенно лучше, чем объединять всю логику в одну команду отправки, так как клиент может независимо создавать сущность, настраивать аудиторию и инициировать доставку.

3.4.2 Контракт реестра шаблонов

Сервис TemplateRegistry предоставляет методы CreateTemplate, UpdateTemplate, GetTemplate, ListTemplates и RenderPreview. Реестр шаблонов используется в двух режимах:

а) как справочник шаблонов и их версий;
б) как сервис предварительного рендеринга содержимого для выбранного канала.

Метод RenderPreview принимает template_id, версию, канал и payload,

после чего находит соответствующий канал в template_version.contents и возвращает отрендеренные subject и body.

3.4.3 Контракты профиля получателя и истории

Сервис ProfileConsentService реализует методы GetRecipientProfile, BatchGetRecipientProfiles и CheckRecipientChannel. Сервис истории NotificationHistoryService реализует ListClientHistory, GetRecipientDeliverySummary и BatchGetRecipientDeliverySummaries.

Пара этих сервисов образует инфраструктурный слой принятия решения о допустимости доставки. profile-consent отвечает на вопрос «можно ли доставлять по каналу и какой destination использовать», а history-writer отвечает на вопрос «не превышена ли допустимая частота доставки в заданном окне времени».

3.5 Реализация рендеринга шаблонов

Фактическая логика выбора шаблона и подстановки параметров устроена следующим образом.

а) Фасад вызывает resolveDeliveryPayload(...).
б) Если во входном payload уже есть subject и один из текстовых атрибутов (body, text, message, content), встроенный контент используется без обращения к template-registry.
в) Если встроенного контента нет, а интеграция включена, фасад вызывает RenderPreview.
г) Реестр шаблонов выбирает версию шаблона, находит контент по каналу и прогоняет subject и body через движок рендеринга.
д) Полученные subject/body возвращаются в фасад и добавляются в общий payload события.

В проекте реализованы два движка рендеринга:

а) TEMPLATE_ENGINE_HANDLEBARS — реальный Handlebars через библиотеку com.github.jknack.handlebars;
б) TEMPLATE_ENGINE_FREEMARKER — упрощённый рендерер SimpleTemplateRenderer, который выполняет простую замену маркеров {key} и {{key}}, но не реализует полный набор возможностей FreeMarker.

Допущение: Текущий enum TEMPLATE_ENGINE_FREEMARKER в кодовой базе не означает подключение полноценного движка Apache FreeMarker. В текущей реализации это упрощённый шаблонизатор строк. Если в окончательной версии ВКР требуется описывать именно FreeMarker как промышленный движок, кодовую реализацию нужно доработать отдельно.

Использование предварительного рендера в фасаде принципиально означает, что подстановка параметров может отработать и до TriggerDispatch. Если subject/body были внедрены при создании события, то sender-сервисам уже не нужен повторный вызов реестра шаблонов: они читают готовый payload и отправляют его дальше.

3.6 Реализация фасадного пайплайна

3.6.1 Создание события

Метод create(...) в JdbcNotificationFacadeUseCase выполняет следующие шаги:

а) проверяет дубликат по client_id + idempotency_key;

б) вычисляет scheduledAt и начальный статус по стратегии;

в) вызывает resolveDeliveryPayload(...);

г) вставляет запись в notification_event;

д) сохраняет аудиторию и, при явном списке, получателей;

е) записывает в outbox событие EventCreated.

Важно, что публикация EventCreated выполняется даже до TriggerDispatch. Именно поэтому sender-сервисы могут реагировать на факт создания события и строить собственные записи планирования отдельно от запуска канальной dispatch-волны.

3.6.2 Запуск доставки

Метод triggerDispatch(...) создаёт запись dispatch, при необходимости снимает snapshot явной аудитории в dispatch_target и затем вызывает enqueueChannelDispatch(...). В зависимости от preferred_channel создаётся outbox-сообщение одного из двух типов:

а) mail_dispatch / MailDispatchRequested;

б) sms_dispatch / SmsDispatchRequested.

Маршрутизация Kafka-топика далее выполняется в OutboxRelayService.topicFor(...). Для mail_dispatch дополнительно анализируется planned_send_at: если время отправки лежит в будущем, сообщение уходит в notification.mail.dispatches.scheduled, иначе сразу в notification.mail.dispatches.

3.7 Реализация sender-сервисов

3.7.1 Почтовый контур

Почтовый сервис содержит два Kafka-консьюмера: один принимает notification.facade.events, второй — notification.mail.dispatches. Оба консьюмера не создают доставку напрямую, а сохраняют сообщение во входящий журнал nf_mail.consumer_inbox_message.

Далее работают два workflow-компонента:

а) NotificationEventWorkflow обрабатывает EventCreated;

б) MailDispatchWorkflow обрабатывает MailDispatchRequested и собственно отправку mail_delivery.

На этапе планирования используется JdbcRecipientDeliveryPolicyEvaluator. Он сначала вызывает profile-consent.checkRecipientChannel(recipientId, CHANNEL_EMAIL), а затем, если включена проверка истории, делает history-writer.getRecipientDeliverySummary(...). При недоступности истории почтовый сервис работает в режиме fail-open и не блокирует доставку только из-за ошибки чтения агрегата истории.

Шлюз SpringMailGateway формирует SimpleMailMessage из payload. Он берёт subject из subject или title, а текст — из body, text, message или content. Если включён режим log-only, фактическая отправка заменяется логированием и доставка считается успешной в рамках тестового контура. Если режим отключён, используется JavaMailSender.

3.7.2 SMS-контур

SMS-сервис повторяет общую схему: принимает события и dispatch-сообщения, пишет их во входящий журнал nf_sms.consumer_inbox_message, затем отдельные workflow-ы создают записи в sms_delivery и выполняют отправку.

Отличие состоит в двух деталях.

а) Планирование в SmsDeliveryPlanService выполняется пакетно по списку получателей.

б) Перед самой отправкой LoggingSmsGateway дополнительно прогоняет batch через SmsProviderGuardService, который ещё раз вызывает profile-consent.

Эта дополнительная SMS-проверка полезна как защитный барьер перед шлюзом, однако она увеличивает число профильных вызовов и должна учитываться при анализе нагрузочного профиля системы.

3.8 Реализация сервиса истории

Сервис history-writer имеет два канала поступления данных:

а) Kafka-консьюмер DeliveryHistoryConsumer, читающий notification.mail.delivery-statuses и notification.sms.delivery-statuses;

б) gRPC-сервис чтения истории и агрегированных summary.

Kafka-слой выполняет только нормализацию входящего статусного события и запись в delivery_history. gRPC-слой не вмешивается в доставку, а предоставляет историю клиентам и другим сервисам. Метод GetRecipientDeliverySummary валидирует recipient_id, нормализует lookback_hours и вычисляет сводку по каналу в заданном временном окне.

3.9 Конфигурация и эксплуатационные ограничения текущей реализации

Текущая кодовая база содержит несколько особенностей, которые необходимо явно зафиксировать в отчёте.

а) В mail- и SMS-сервисах присутствуют режимы тестовой отправки, когда внешнее взаимодействие заменяется логированием.

б) Для части gRPC-клиентов и Kafka-подключений в демонстрационном профиле используется plaintext.

в) В профиле получателя предусмотрен канал push, но отдельный push-sender в репозитории отсутствует.

г) Топик notification.facade.dispatches публикуется фасадом, однако в текущей реализации не участвует в прикладной обработке sender-сервисов.

Эти особенности не делают систему некорректной, но определяют границы того, что можно честно утверждать о текущем программном результате.

3.10 Выводы по разделу

Реализация подтверждает жизнеспособность выбранной архитектуры: все критические сущности отражены в явной модели данных, межсервисное взаимодействие описано protobuf-контрактами, а доставка выполняется через воспроизводимый Kafka-пайплайн с outbox/inbox-механизмами. Особое значение имеют три факта: рендер шаблона может происходить уже на этапе создания события, sender-сервисы работают с устойчиво сохранёнными входящими сообщениями, а почтовый и SMS-контуры имеют разную глубину проверок получателя. Эти различия принципиальны для корректного понимания поведения системы под нагрузкой и при восстановлении после сбоев.

4 Развёртывание, эксплуатация и тестирование

4.1 Организация развёртывания

Для воспроизводимого запуска проекта используется контейнерный контур на базе Docker Compose, размещённый в каталоге deploy. В нём поднимаются инфраструктурные компоненты платформы: PostgreSQL, Kafka, Redis, Prometheus, Grafana и вспомогательные контейнеры. Такой способ развёртывания важен не только как удобство повторяемого запуска, но и как инженерный механизм повторяемой сборки стенда, пригодного для проверки межсервисных сценариев [28].

Контейнерный слой даёт следующие преимущества:

а) фиксирует набор версий инфраструктурных зависимостей;

б) позволяет быстро приводить стенд к воспроизводимому состоянию;

в) разделяет инфраструктурные роли по контейнерам;

г) облегчает очистку и повторную инициализацию окружения перед серией испытаний.

Прикладные сервисы платформы используют единый набор транспортов и хранилищ, но при этом сохраняют собственные схемы в PostgreSQL и собственные циклы фоновой обработки. Это позволяет запускать или перезапускать отдельный сервис без необходимости пересоздавать весь прикладной слой.

4.2 Автоматизация выкладки

В проекте реализован workflow автоматизированной выкладки через GitHub Actions. Его назначение заключается не в сложной orchestration-логике, а в обеспечении повторяемой процедуры обновления окружения:

а) извлечение актуального состояния репозитория;

б) запуск команды синхронизации Docker Compose;

в) пересоздание изменённых контейнеров;

г) сохранение единообразного сценария обновления стенда.

Даже при сравнительно простом составе сервиса эта автоматизация важна, поскольку исключает ручное рассогласование конфигураций и уменьшает вероятность ошибки при повторяемых тестовых и

демонстрационных развёртываниях [29; 28].

4.3 Средства наблюдаемости

Наблюдаемость платформы построена на сочетании Spring Boot Actuator, Micrometer, Prometheus и Grafana. Каждый сервис экспортирует метрики и health-check-и, а Prometheus централизованно собирает их с прикладных endpoint-ов [20; 21; 22].

В рамках данной работы использовались четыре группы метрик:

а) серверные gRPC-метрики по интенсивности и времени обработки запросов;
б) Kafka producer-метрики, прежде всего семейство spring_kafka_template_seconds;
в) Kafka consumer-метрики, прежде всего семейство spring_kafka_listener_seconds;
г) системные метрики процессов и контейнеров, включая память, CPU и число потоков.

Наличие этих метрик важно по двум причинам. Во-первых, можно увидеть, проходит ли сообщение через все проектные стадии: приём команды, публикацию в Kafka, приём консьюмером, канальную доставку и запись истории. Во-вторых, появляется возможность сопоставлять архитектурные решения с их реальным эксплуатационным следом, например с ростом задержки конкретного фасадного метода или с увеличением частоты вызова профильного сервиса.

4.4 Цели тестирования

Тестирование платформы было ориентировано не на демонстрацию абстрактной производительности, а на подтверждение корректности сквозного жизненного цикла уведомления, устойчивости к повторным вызовам и наблюдаемости состояния системы.

Основные цели испытаний:

а) подтвердить корректную обработку команды создания события и запуска доставки;
б) проверить, что отложенная почтовая отправка переводится через планировщик в рабочий контур без потери сообщения;

в) убедиться, что профиль получателя и история действительно участвуют в принятии решения о доставке;
г) зафиксировать корректную публикацию статусных сообщений и их появление в истории;
д) проверить восстановление после временных ошибок и повторных попыток;
е) подтвердить, что ключевые стадии пайплайна отражаются в метриках.

4.5 Методика проверки

Проверка проводилась как совокупность модульных, интеграционных и сквозных сценариев. Для данной платформы особенно важны именно сквозные проверки, поскольку большая часть риска лежит не в отдельном методе, а в переходе сообщения между границами сервисов.

В качестве основных сквозных сценариев рассматривались:

а) создание события с явной аудиторией и последующая публикация EventCreated;
б) запуск email-доставки через TriggerDispatch и появление сообщения в notification.mail.dispatches;
в) запуск SMS-доставки через TriggerDispatch и появление сообщения в notification.sms.dispatches;
г) отложенная почтовая отправка через notification.mail.dispatches.scheduled и scheduler-delivery;
д) отклонение доставки при отрицательном ответе profile-consent;
е) отклонение доставки по превышению лимита в сервисе истории;
ж) повторный вызов команд с тем же ключом идемпотентности;
и) запись финальных и промежуточных статусов в history-writer.

4.6 Контрольные критерии успешности

Для каждого сценария использовались не только функциональные ожидания, но и внутренние критерии согласованности, приведённые в таблице 6.

Таблица 6 – Критерии успешности испытаний

Проверяемое свойство	**Ожидаемый признак корректной работы**
Идемпотентность команды создания	Повторный вызов CreateNotificationEvent не создаёт второй notification_event.
Надёжность публикации	Доменное изменение сопровождается записью в outbox_message, а не прямой нефиксируемой отправкой в Kafka.
Безопасный приём Kafka-сообщения	Канальный сервис сначала сохраняет сообщение во входящий журнал, а не обрабатывает его только в памяти.
Корректность policy check	Решение о доставке зависит от profile-consent и, при включении опции, от history summary.
Отработка повторных попыток	Временная ошибка переводит доставку в RETRY, а не в безусловную потерю.
Трассируемость результата	Финальный либо промежуточный статус появляется в delivery_history.
Наблюдаемость	В метриках видны входной запрос, публикация в Kafka и работа listener-ов.

4.7 Проверка отказоустойчивости

Особое внимание уделялось не только позитивным сценариям, но и штатной реакции на временный отказ зависимости. Для этого анализировались следующие ситуации:

а) временная недоступность Kafka для outbox relay;
б) ошибка канального шлюза при отправке email или SMS;
в) перезапуск sender-сервиса между приёмом Kafka-сообщения и завершением workflow;
г) повторная доставка одного и того же транспортного сообщения;
д) невозможность получить history summary;
е) отказ profile-consent по бизнес-правилу.

Результат считался корректным, если система либо откладывала задачу на повторную попытку, либо завершала её явным статусом SKIPPED или FAILED, но не теряла состояние молча и не порождала неконтролируемых дублей.

4.8 Нагрузочный прогон как проверка качества пайплайна

Для практической верификации поведения системы под непрерывным потоком сообщений был выполнен стендовый нагрузочный прогон с

использованием custom-loader и последующим анализом эксплуатационных метрик в Grafana. Важно подчеркнуть, что целью этого прогона была не публикация абсолютного промышленного benchmark-а, а проверка согласованности пайплайна и выявление относительных узких мест.

В качестве артефактов качества были сохранены графики:

а) серверного gRPC RPS;

б) задержки ключевых gRPC-вызовов;

в) задержки публикации сообщений в Kafka;

г) интенсивности потребления Kafka-сообщений.

Соответствующие скриншоты вынесены в приложение с артефактами наблюдаемости, чтобы основная глава не превращалась в набор картинок без пояснения смысла.

4.9 Качественный анализ метрик

Анализ артефактов наблюдаемости позволяет сделать несколько содержательных выводов.

Во-первых, интенсивность входных gRPC-вызовов фасада и профильного сервиса показывает, что проверка профиля получателя действительно входит в критический путь sender-процессов. Это подтверждает архитектурное предположение о том, что profile-consent является одним из ключевых сервисов принятия решения.

Во-вторых, графики задержек показывают, что наиболее дорогим участком стендового пайплайна остаётся фасадный этап приёма и подготовки события. Это согласуется с реализацией, поскольку именно здесь выполняются идемпотентная проверка, работа с таблицами события и аудитории, а также, при необходимости, вызов рендера шаблона.

В-третьих, producer- и listener-метрики Kafka подтверждают, что транспортный слой наблюдаем и поддаётся анализу не только по логам, но и по численным сериям Micrometer. Следовательно, архитектурный выбор в пользу Kafka не ограничился логическим проектированием: он действительно сопровождается измеримым эксплуатационным следом.

В-четвёртых, различие между скоростью приёма событий и скоростью появления финальных записей истории подтверждает асинхронную природу контура. История не обязана обновляться синхронно с принятием команды; напротив, отставание здесь является ожидаемым свойством многостадийной

обработки.

4.10 Интерпретация результатов

С инженерной точки зрения нагрузочный прогон дал не одну цифру, а набор практически полезных выводов:

а) событийный пайплайн работоспособен и наблюдаем от входного gRPC-запроса до записи статуса доставки;
б) фасадный контур и проверки получателя формируют заметную часть задержки;
в) Kafka-контур успешно используется как инструмент декомпозиции обработки, а не как скрытый источник состояния;
г) sender-сервисы и history-writer оставляют в системе измеримый след, по которому можно локализовать рассогласование или перегрузку.

Именно в этом состоит ценность испытаний для ВКР: они подтверждают не абстрактный тезис о «быстрой системе», а инженерную состоятельность выбранной архитектуры и её пригодность к дальнейшей оптимизации на основании реальных метрик.

4.11 Выводы по разделу

Развёртывание и тестирование показали, что реализованная платформа воспроизводимо запускается в контейнерном стенде, поддерживает автоматизированную выкладку, экспортирует эксплуатационные метрики и допускает сквозную проверку жизненного цикла уведомления. Нагрузочный прогон в данном контексте выступил не как самоцель, а как средство подтвердить согласованность потоков данных и обнаружить участки, требующие дальнейшей инженерной оптимизации. В совокупности это подтверждает эксплуатационную зрелость реализованного программного результата.

5 Технико-экономическая оценка и анализ эксплуатационных рисков

5.1 Подход к технико-экономической оценке

Экономическая часть для рассматриваемой платформы должна оценивать не только стоимость написания кода, но и эксплуатационный эффект архитектурных решений. Система уведомлений является инфраструктурным компонентом, поэтому её ценность проявляется прежде всего в снижении операционных потерь: потерь уведомлений, затрат на ручную диагностику, стоимости инцидентов и времени восстановления.

В рамках данной ВКР корректно использовать качественную технико-экономическую оценку. Для строгого количественного расчёта по нормативной вузовской методике необходимы утверждённые исходные показатели: ставки исполнителей, нормативы трудоёмкости, тарифы на инфраструктуру, стоимость владения внешними шлюзами, регламентированные коэффициенты накладных расходов и принятая кафедрой схема приведения затрат.

Допущение: В текущем репозитории и сопроводительных материалах отсутствуют утверждённые кафедрой числовые исходные данные для полного калькуляционного расчёта себестоимости и экономического эффекта. Поэтому раздел выполнен как качественная технико-экономическая оценка с явным анализом факторов стоимости, эффекта и рисков, без вымышленных денежных величин.

5.2 Основные статьи затрат

Для платформы мультиканальной доставки уведомлений можно выделить четыре группы затрат.

5.2.1 Затраты на разработку

В эту группу входят:

а) проектирование доменной модели события, запуска доставки и канальной доставки;

б) реализация gRPC-контрактов и Kafka-пайплайна;

в) создание миграций PostgreSQL и Redis-модели профиля;

г) разработка механизмов идемпотентности, outbox/inbox и повторных

попыток;

д) реализация sender-сервисов и интеграции с историей.

На начальном этапе такие затраты выше, чем у упрощённого монолитного сервиса с прямой синхронной отправкой. Однако это увеличение затрат компенсируется снижением технического долга и уменьшением стоимости последующих изменений.

5.2.2 Инфраструктурные затраты

К инфраструктурной части относятся:

а) вычислительные ресурсы прикладных сервисов;

б) PostgreSQL как устойчивое хранилище доменного состояния;

в) Redis как оперативное хранилище профилей;

г) Kafka как межсервисный транспорт;

д) Prometheus и Grafana как средства наблюдаемости.

По сравнению с централизованной схемой количество компонентов возрастает, однако каждый из них закрывает отдельный класс инженерных рисков: Kafka уменьшает связанность и позволяет управлять асинхронной нагрузкой, Redis ускоряет доступ к профилю получателя, а Prometheus и Grafana сокращают стоимость диагностики.

5.2.3 Эксплуатационные затраты

В эксплуатационные затраты входят:

а) сопровождение контейнерного контура;

б) контроль миграций схем данных;

в) поддержка релизного процесса и стендов;

г) мониторинг инцидентов и разбор проблемных доставок;

д) администрирование секретов и политик безопасности.

Именно на этой стадии архитектурные решения начинают давать экономический эффект. Наличие истории, явных статусов и метрик сокращает затраты на поиск причины проблемы и уменьшает число ручных операций при восстановлении обработки.

5.3 Экономический эффект от архитектурных решений

5.3.1 Эффект от разделения жизненного цикла

Разделение на событие, запуск доставки и отдельную канальную доставку позволяет избежать целого класса ошибок, когда повторный запуск или частичная неуспешность ломают общее состояние уведомления. С экономической точки зрения это снижает стоимость сопровождения, поскольку каждая стадия имеет собственные таблицы, собственные статусы и отдельную точку анализа.

5.3.2 Эффект от outbox/inbox-подхода

Паттерны outbox и inbox увеличивают сложность реализации, но уменьшают риск тихой потери сообщений и трудоёмкость восстановления после сбоя. Для сервиса уведомлений это особенно важно: стоимость одной потерянной транзакционной коммуникации может значительно превышать стоимость дополнительной технической логики в коде.

5.3.3 Эффект от централизованного профиля получателя

Выделение profile-consent как отдельного сервиса устраняет дублирование логики проверки согласий, блокировок и каналов в sender-сервисах. Экономически это снижает стоимость развития новых каналов и уменьшает риск рассинхронизации бизнес-правил между разными отправителями.

5.3.4 Эффект от наблюдаемости

Метрики Kafka-публикации, listener-обработки и gRPC-вызовов создают прямой эксплуатационный эффект: инженер может определить, где именно возникла проблема, не прибегая к догадкам и трудоёмкому анализу всех журналов вручную. Это уменьшает время локализации инцидента и тем самым снижает операционные затраты.

5.4 Сравнительная оценка вариантов

В таблице 7 приведено качественное сравнение двух вариантов: упрощённого централизованного контура и реализованной микросервисной

архитектуры.

Таблица 7 – Качественное сравнение архитектурных вариантов

Критерий	**Централизованный сервис**	**Реализованная архитектура**
Начальная стоимость разработки	Ниже	Выше
Стоимость локализации сбоя	Выше	Ниже
Стоимость расширения новым каналом	Выше из-за сильной связанности	Ниже за счёт выделенных сервисных границ
Риск потери сообщения при сбое	Выше	Ниже благодаря outbox/inbox
Стоимость поддержки истории и аудита	Выше	Ниже за счёт отдельного history-сервиса
Предсказуемость поведения при повторе	Ниже	Выше благодаря идемпотентности и уникальным ограничениям

Таким образом, реализованное решение экономически оправдано не в логике «минимальная цена входа», а в логике уменьшения будущих эксплуатационных и организационных потерь.

5.5 Анализ рисков информационной безопасности

Для платформы уведомлений критичны не только экономические, но и информационно-безопасностные риски, поскольку система оперирует данными о получателях, каналами доставки и служебными идентификаторами. Базовые меры защиты межсервисного взаимодействия в таких системах опираются на шифрование транспортного канала и контролируемую сервисную аутентификацию [26; 24; 25].

Ключевые риски и меры снижения представлены в таблице 8.

Отдельно необходимо учитывать, что эксплуатационная стоимость инцидента безопасности для подобной платформы обычно выше стоимости превентивных мер. Даже если меры защиты усложняют инфраструктуру, они экономически оправданы за счёт снижения риска компрометации каналов и получательских данных.

Таблица 8 – Ключевые риски информационной безопасности

Риск	**Последствие**	**Меры снижения**
Несанкционированный вызов фасада	Запуск ложных уведомлений, нарушение бизнес-процессов	Сервисная аутентификация, проверка client_id, аудит вызовов
Перехват межсервисного трафика	Компрометация payload и служебных идентификаторов	TLS, сервисные сертификаты, сегментация сети
Утечка секретов шлюзов	Несанкционированная отправка через внешних провайдеров	Внешнее хранилище секретов, ротация, ограничение доступа
Чтение данных профиля из Redis без контроля доступа	Утечка контактов и признаков согласия	Закрытие прямого доступа, использование только через profile-consent
Избыточное логирование payload	Попадание персональных данных в журналы	Маскирование чувствительных полей, политика логирования, контроль retention

5.6 Влияние архитектуры на стоимость сопровождения

С точки зрения сопровождения наиболее экономически значимыми свойствами системы являются:

а) отдельный сервис истории, позволяющий разбирать инциденты по фактам, а не по косвенным признакам;

б) явные доменные статусы и таблицы попыток отправки;

в) централизованный profile-consent, уменьшающий дублирование логики в sender-сервисах;

г) возможность тестировать и дорабатывать отдельный контур без переписывания всей системы;

д) наблюдаемость каждого этапа через метрики Kafka и gRPC.

Именно эти свойства уменьшают полную стоимость владения в среднесрочной перспективе. Архитектурная дисциплина требует больших усилий на старте, но затем снижает цену сопровождения, потому что уменьшает хаотичность поведения системы.

5.7 Выводы по разделу

Выполненная технико-экономическая оценка показывает, что

реализованная платформа имеет более высокую стартовую сложность по сравнению с простым централизованным сервисом, но экономически выигрывает на этапе эксплуатации. Основной эффект достигается за счёт уменьшения потерь сообщений, снижения трудоёмкости диагностики, упрощения расширения новыми каналами и повышения управляемости межсервисного взаимодействия. Дополнительный вклад в экономическую целесообразность вносит архитектура безопасности: затраты на защитные меры оправданы тем, что предотвращают существенно более дорогие инциденты, связанные с утечкой данных и несанкционированной доставкой.

ЗАКЛЮЧЕНИЕ

Выпускная квалификационная работа была посвящена проектированию и реализации отказоустойчивой распределённой платформы доставки уведомлений, основанной на событийном межсервисном взаимодействии, явном управлении жизненным циклом уведомления и контролируемой обработке канальных доставок.

Поставленная цель работы достигнута. В ходе выполнения ВКР:

а) проведён анализ предметной области платформ уведомлений и практик проектирования распределённых систем;

б) сформулированы функциональные и нефункциональные требования к системе, включая требования к надёжности, идемпотентности, наблюдаемости и управляемости обработки;

в) спроектирована архитектура, разделяющая событие уведомления, запуск доставки и отдельную канальную доставку;

г) реализован фасадный gRPC-контур управления событиями и аудиторией;

д) реализован сервис шаблонов с версионированием и предварительным рендерингом содержимого;

е) реализован сервис профиля получателя на базе Redis с gRPC-интерфейсом проверки каналов;

ж) реализованы канальные сервисы электронной почты и SMS с отдельными входящими и исходящими журналами, таблицами доставок и попыток;

и) реализован сервис отложенной почтовой отправки;

к) реализован сервис истории доставки, агрегирующий статусные события из Kafka;

л) подготовлен контейнерный контур развёртывания, средства наблюдаемости и методика стендовой верификации системы.

Практическая значимость результатов заключается в том, что разработанная система подтверждает применимость архитектурного подхода, в котором надёжность достигается не распределённой транзакцией, а сочетанием локальных транзакций, паттернов outbox/inbox, идемпотентности, повторных попыток и явной фиксации промежуточных состояний. Для платформы уведомлений это особенно важно, поскольку ошибка редко

проявляется как единичное исключение в коде; чаще она проявляется как потеря события, повторная отправка, отсутствие истории или некорректная маршрутизация получателя.

Важным результатом работы стало также подтверждение того, что архитектура остаётся наблюдаемой. Метрики gRPC, producer- и listener-метрики Kafka, а также артефакты нагрузочного прогона позволяют анализировать прохождение сообщения по всей цепочке: от приёма команды до записи результата в историю. Это делает платформу не только функционально реализованной, но и пригодной к инженерному сопровождению.

При этом ВКР честно фиксирует границы текущего программного результата. В репозитории реализованы каналы email и SMS; канал push присутствует в модели профиля получателя как направление расширения, но отдельный sender-сервис для него пока не реализован. Аналогично, в демонстрационном профиле часть механизмов промышленной защиты транспорта и аутентификации ещё требует доведения до production-уровня.

Перспективными направлениями развития разработанной платформы являются:

а) реализация полноценного push-канала на базе уже подготовленной модели профиля;
б) завершение промышленного контура безопасности с обязательным TLS, сервисной аутентификацией и централизованным управлением секретами;
в) дальнейшая оптимизация фасадного участка обработки, который по метрикам остаётся наиболее чувствительным к росту нагрузки;
г) развитие батчевых проверок профиля и истории для уменьшения стоимости массовых dispatch-сценариев;
д) расширение набора эксплуатационных сценариев и автоматизированных тестов отказоустойчивости.

Таким образом, результат работы представляет собой инженерно обоснованную реализацию распределённой платформы уведомлений, в которой сочетаются контролируемая канальная обработка, централизованная проверка профиля получателя, событийная интеграция через Kafka и отдельный контур истории. Полученный программный результат может служить основой для дальнейшего развития системы и её доведения до

промышленного уровня эксплуатации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Apache Kafka Documentation. — URL: https://kafka.apache.org/documentation/ (дата обращения 25.03.2026).

2. Kleppmann M. Designing Data-Intensive Applications. — Sebastopol : O'Reilly Media, 2017.

3. Newman S. Building Microservices. — 2-е изд. — Sebastopol : O'Reilly Media, 2021.

4. Richardson C. Microservices Patterns. — Shelter Island : Manning Publications, 2018.

5. Nygard M. T. Release It! Design and Deploy Production-Ready Software. — 2-е изд. — Raleigh : Pragmatic Bookshelf, 2018.

6. Apache Kafka: Design – Message Delivery Semantics. — URL: https://kafka.apache.org/40/design/design/ (дата обращения 25.03.2026).

7. Apache Kafka: Producer Configs. — URL: https://kafka.apache.org/40/configuration/producer-configs/ (дата обращения 25.03.2026).

8. PostgreSQL Documentation. — URL: https://www.postgresql.org/docs/current/ (дата обращения 25.03.2026).

9. PostgreSQL: SELECT. — URL: https://www.postgresql.org/docs/current/sql-select.html (дата обращения 25.03.2026).

10. Spring Framework JDBC: Choosing an Approach for JDBC Database Access. — URL: https://docs.spring.io/spring-framework/reference/data-access/jdbc/choose-style.html (дата обращения 25.03.2026).

11. gRPC Documentation. — URL: https://grpc.io/docs/ (дата обращения 25.03.2026).

12. Redgate Flyway Documentation. — URL: https://documentation.red-gate.com/flyway (дата обращения 11.04.2026).

13. Redis Documentation. — URL: https://redis.io/docs/ (дата обращения 11.04.2026).

14. The C4 Model for Software Architecture. — URL: https://c4model.com/ (дата обращения 25.03.2026).

15. Structurizr DSL Documentation. — URL: https://docs.structurizr.com/dsl (дата обращения 11.04.2026).

16. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — URL: https://docs.cntd.ru/document/1200157208 (дата обращения 25.03.2026).

17. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. — URL: https://docs.cntd.ru/document/1200063713 (дата обращения 25.03.2026).

18. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — URL: https://docs.cntd.ru/document/1200161674 (дата обращения 25.03.2026).

19. RabbitMQ Reliability Guide. — URL: https://www.rabbitmq.com/docs/reliability (дата обращения 25.03.2026).

20. Spring Boot Actuator. — URL: https : / / docs . spring . io / spring - boot / reference / actuator/ (дата обращения 25.03.2026).

21. Micrometer Documentation. — URL: https://docs.micrometer.io/micrometer/reference/ (дата обращения 25.03.2026).

22. Prometheus Documentation: Overview. — URL: https://prometheus.io/docs/introduction/overview/ (дата обращения 25.03.2026).

23. OpenTelemetry Documentation. — URL: https://opentelemetry.io/docs/ (дата обращения 25.03.2026).

24. RFC 6749: The OAuth 2.0 Authorization Framework. — URL: https://www.rfc-editor.org/rfc/rfc6749 (дата обращения 25.03.2026).

25. RFC 7519: JSON Web Token (JWT). — URL: https://www.rfc-editor.org/rfc/rfc7519 (дата обращения 25.03.2026).

26. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. — URL: https://www.rfc-editor.org/rfc/rfc8446 (дата обращения 25.03.2026).

27. Spring Boot Reference Documentation. — URL: https://docs.spring.io/spring-boot/reference/ (дата обращения 25.03.2026).

28. Docker Compose Documentation. — URL: https://docs.docker.com/compose/ (дата обращения 25.03.2026).

29. GitHub Actions Documentation. — URL: https://docs.github.com/actions (дата обращения 25.03.2026).

ПРИЛОЖЕНИЕ А
Контракты сервисного взаимодействия

0.8 Контракт фасада уведомлений

Фасадный gRPC-сервис NotificationFacade представляет внешний прикладной API системы. Его методы сгруппированы по назначению в таблице А.1.

Таблица А.1 – Методы сервиса NotificationFacade

Метод	**Назначение**
CreateNotificationEvent	Создание события уведомления с ключом идемпотентности, стратегией отправки и payload.
UpdateNotificationEvent	Изменение редактируемых полей события по update_mask.
CancelNotificationEvent	Перевод события в состояние отмены.
GetNotificationEvent	Получение текущего состояния события.
ListNotificationEvents	Постраничный просмотр событий клиента.
SetAudience	Полная замена описания аудитории события.
AddRecipients	Добавление получателей к явной аудитории.
RemoveRecipients	Удаление получателей из явной аудитории.
GetAudience	Чтение текущей аудитории события.
TriggerDispatch	Создание запуска доставки и публикация канальной dispatch-команды.
GetDispatch	Получение информации о конкретном запуске доставки.
ListDispatches	Получение списка запусков по событию.

Ключевые входные атрибуты фасадного API:

а) client_id — идентификатор клиента, связанный с вызывающей стороной;
б) idempotency_key — ключ защиты от повторной обработки;
в) template_id и template_version — ссылка на шаблон;
г) preferred_channel — предпочитаемый канал доставки;
д) payload — словарь параметров для рендера и отправки;
е) audience — описание аудитории или явный список получателей;
ж) strategy — немедленная либо отложенная отправка.

0.9 Контракт реестра шаблонов

Сервис TemplateRegistry реализует методы, приведённые в таблице А.2.

Таблица А.2 – Методы сервиса TemplateRegistry

Метод	Назначение
CreateTemplate	Создание карточки шаблона и начальной версии.
UpdateTemplate	Добавление новой версии или изменение активной версии шаблона.
GetTemplate	Получение полного описания шаблона и версий.
ListTemplates	Постраничный просмотр шаблонов клиента.
RenderPreview	Рендер subject/body для выбранной версии и канала на основании payload.

Практически наиболее значимым для текущей архитектуры является именно RenderPreview, поскольку он используется фасадом для инъекции готового текста в payload события ещё до канальной отправки.

0.10 Контракт профиля получателя

Сервис ProfileConsentService предоставляет инфраструктурный API проверки получателя и канала. Его методы приведены в таблице А.3.

Таблица А.3 – Методы сервиса ProfileConsentService

Метод	Назначение
GetRecipientProfile	Получение полного профиля одного получателя.
BatchGetRecipientProfiles	Получение профилей группы получателей.
CheckRecipientChannel	Проверка допустимости канала, возврат allowed, reason_code, preferred_channel и destination.

Именно CheckRecipientChannel используется sender-сервисами на критическом пути планирования доставки.

0.11 Контракт сервиса истории

Сервис NotificationHistoryService выполняет чтение агрегированной истории и сводок, как показано в таблице А.4.

В sender-сервисах summary используется как дополнительное бизнес-ограничение на частоту доставки в пределах заданного окна времени.

Таблица А.4 – Методы сервиса NotificationHistoryService

Метод	**Назначение**
ListClientHistory	Постраничный просмотр истории по клиенту.
GetRecipientDeliverySummary	Получение сводки по одному получателю за окно времени.
BatchGetRecipientDeliverySummaries	Получение набора сводок по группе получателей.

0.12 Совместимость и эволюция контрактов

Все перечисленные сервисы используют protobuf-контракты, что даёт два важных инженерных свойства:

а) строгую типизацию сообщений и единый машинно-читаемый источник API;

б) безопасную эволюцию за счёт добавления новых необязательных полей без нарушения обратной совместимости существующих клиентов [11].

ПРИЛОЖЕНИЕ Б
Kafka-топики и поток сообщений

0.13 Основные топики платформы

Kafka используется как транспорт асинхронного обмена между фасадом, планировщиком, sender-сервисами и сервисом истории. Актуальная карта топиков приведена в таблице Б.1.

Таблица Б.1 – Назначение Kafka-топиков

Топик	**Продюсер**	**Консьюмер**	**Основные типы событий**
notification.facade.events	Notification Facade	Mail Sender, SMS Sender	EventCreated
notification.facade.dispatches	Notification Facade	Нет выделенного бизнес-потребителя в текущем коде	События по dispatch
notification.mail.dispatches	Notification Facade, Scheduler Delivery	Mail Sender	MailDispatchRequested
notification.mail.dispatches.scheduled	Notification Facade	Scheduler Delivery	MailDispatchRequested с будущим planned_send_at
notification.sms.dispatches	Notification Facade	SMS Sender	SmsDispatchRequested
notification.mail.delivery-statuses	Mail Sender	History Writer	MailDeliveryStatusChanged
notification.sms.delivery-statuses	SMS Sender	History Writer	SmsDeliveryStatusChanged

0.14 Формат интеграционного сообщения

Сообщения, публикуемые из outbox, имеют единый логический конверт:

а) outboxId — идентификатор записи выходящего журнала;

б) aggregateType — тип агрегата, например notification_event, mail_dispatch, sms_delivery;

в) aggregateId — идентификатор агрегата;

г) eventType — прикладной тип события;

д) payload — полезная нагрузка бизнес-события;

е) headers — служебные заголовки, включая message_id и дополнительные идентификаторы;

ж) createdAt — время создания outbox-записи.

Унификация формата позволяет sender-сервисам и history-writer использовать одинаковую схему десериализации и входящего хранения, а также делает трассировку сообщений устойчивой к расширению числа событий.

0.15 Поток EventCreated

Поток событий создания уведомления организован следующим образом:

а) фасад создаёт запись notification_event;

б) в outbox добавляется сообщение EventCreated типа notification_event;

в) relay фасада публикует сообщение в notification.facade.events;

г) Mail Sender и SMS Sender читают сообщение, сохраняют его во входящий журнал и далее строят собственные записи доставки только для своего канала.

Такой подход позволяет sender-сервисам реагировать на факт создания события, не завися от немедленного вызова TriggerDispatch.

0.16 Поток TriggerDispatch

После вызова TriggerDispatch фасад создаёт запись dispatch, snapshot аудитории и канальную outbox-команду:

а) для email — MailDispatchRequested;

б) для SMS — SmsDispatchRequested.

Если email-запуск отложен, OutboxRelayService направляет сообщение в notification.mail.dispatches.scheduled; если момент отправки уже наступил, оно сразу публикуется в notification.mail.dispatches. Для SMS отдельного отложенного топика в текущей реализации нет.

0.17 Поток статусных событий

После обработки доставки sender-сервисы публикуют статусные сообщения через собственные outbox-журналы:

а) Mail Sender формирует MailDeliveryStatusChanged и публикует его в notification.mail.delivery-statuses;

б) SMS Sender формирует SmsDeliveryStatusChanged и публикует его в notification.sms.delivery-statuses;

в) History Writer читает оба топика и нормализует запись в delivery_history.

За счёт такой схемы сервис истории полностью отделён от sender-логики и не вмешивается в принятие решения о доставке. Он работает как отдельный асинхронный потребитель фактов доставки.

0.18 Архитектурные свойства Kafka-слоя

Использование Kafka в данной системе обеспечивает:

а) разделение времени приёма команды и времени фактической канальной доставки;

б) возможность повторной обработки без глобальной транзакции;

в) масштабирование producer- и consumer-частей независимо друг от друга;

г) явную наблюдаемость через producer- и listener-метрики Micrometer.

При этом Kafka не заменяет прикладную идемпотентность. Модель доставки системы остаётся at-least-once, поэтому защита от дублей реализуется в прикладном слое через уникальные ограничения и входящие журналы [1; 6].

ПРИЛОЖЕНИЕ В
Модель данных и хранилища

0.19 Схемы PostgreSQL

Распределение основных таблиц по схемам PostgreSQL приведено в таблице В.1.

Таблица В.1 – Основные схемы и таблицы PostgreSQL

Схема	Таблицы	Назначение
nf_fac	notification_event, event_audience, event_recipient, dispatch, dispatch_target, outbox_message	Управление жизненным циклом события и запусков доставки.
nf_tpl	template, template_version	Хранение карточек шаблонов, активной версии и канального содержимого.
nf_sched	scheduled_delivery_task	Хранение отложенных почтовых задач и их состояний публикации.
nf_mail	consumer_inbox_message, recipient_mail_settings, mail_delivery, mail_delivery_attempt, outbox_message	Почтовый входящий журнал, доставки, попытки и статусный outbox.
nf_sms	consumer_inbox_message, sms_delivery, sms_delivery_attempt, outbox_message	SMS-входящий журнал, доставки, попытки и статусный outbox.
nf_hist	delivery_history	История статусов доставки по каналам.

0.20 Ключевые ограничения и идентификаторы

Наиболее важные ограничения реляционной модели:

а) уникальность события фасада по client_id + idempotency_key;

б) уникальность запуска доставки по event_id + idempotency_key;

в) уникальность фактической доставки по комбинации запуска, получателя и канала;

г) уникальность записей о попытках по номеру попытки для конкретной доставки;

д) идемпотентность записи истории по outbox_id.

Эти ограничения закрывают два класса риска: появление дублей при сетевом повторе и нарушение согласованности при конкурентной обработке нескольких batch-потоков.

0.21 Служебные журналы

С точки зрения архитектуры хранения особое значение имеют две группы таблиц:

а) outbox_message во фасаде и sender-сервисах;

б) consumer_inbox_message в sender-сервисах.

Таблицы outbox содержат:

а) тип агрегата;

б) идентификатор агрегата;

в) тип события;

г) payload и headers в JSON-виде;

д) время создания;

е) признаки аренды, повтора и подтверждения публикации.

Таблицы inbox содержат:

а) идентификатор входящего сообщения;

б) тип агрегата и тип события;

в) полезную нагрузку;

г) статус обработки (NEW, PROCESSING, PROCESSED, FAILED);

д) время приёма и время завершения.

0.22 Redis-модель профиля получателя

В profile-consent используется Redis-хеш. Состав полей приведён в таблице В.2.

0.23 Связь между уровнями данных

Архитектура хранения выстроена так, чтобы каждый уровень решал собственную задачу:

а) PostgreSQL хранит устойчивое доменное состояние, журналы и историю;

б) Redis хранит оперативный профиль получателя;

в) Kafka переносит состояние между сервисами в виде событийных

Таблица В.2 – Поля профиля получателя в Redis

Поле	**Смысл**
active	Признак активности получателя.
preferred_channel	Предпочитаемый канал коммуникации.
email.enabled	Разрешён ли email-канал.
email.blacklisted	Заблокирован ли email-канал.
email.destination	Адрес электронной почты.
sms.enabled	Разрешён ли SMS-канал.
sms.blacklisted	Заблокирован ли SMS-канал.
sms.destination	Номер телефона.
push.enabled	Разрешён ли push-канал.
push.blacklisted	Заблокирован ли push-канал.
push.destination	Идентификатор push-адресата или устройства.
updated_at	Время последнего обновления профиля.

сообщений.

Такое распределение ролей позволяет не смешивать транзакционные данные, кэш получателя и транспорт межсервисного обмена в одном технологическом слое.

ПРИЛОЖЕНИЕ Г

Артефакты наблюдаемости и метрик качества

В данном приложении приведены графические артефакты метрик, использованные в работе как подтверждение наблюдаемости системы. Они не являются самостоятельным доказательством промышленной производительности, а служат визуальным свидетельством того, что ключевые стадии пайплайна — gRPC-вызовы, публикация в Kafka и обработка listener-ами — действительно измеряются и сопоставимы между собой.

Рисунок Г.1 – Интенсивность серверных gRPC-вызовов в ходе стендового прогона

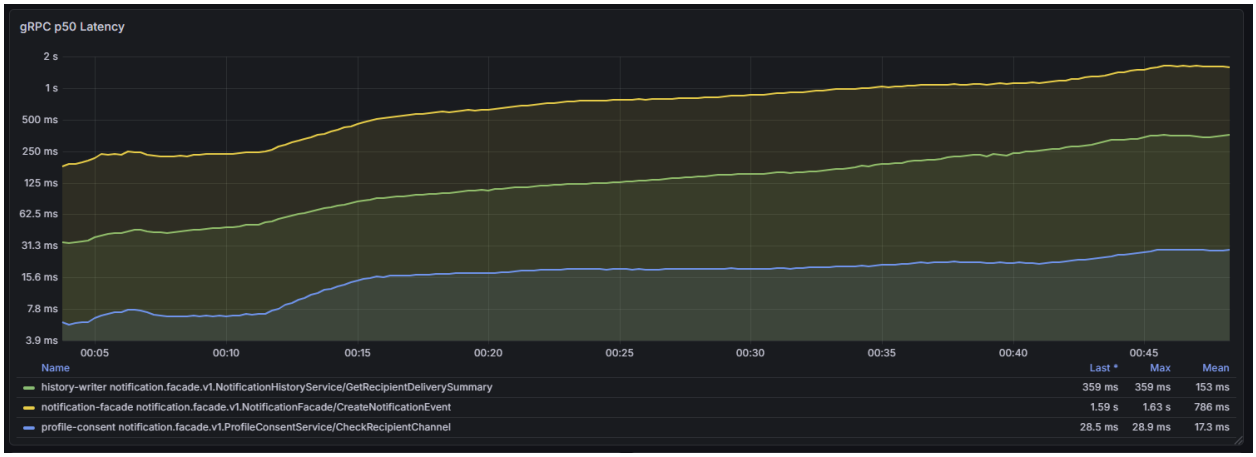


Рисунок Г.2 – Медианная задержка ключевых gRPC-вызовов

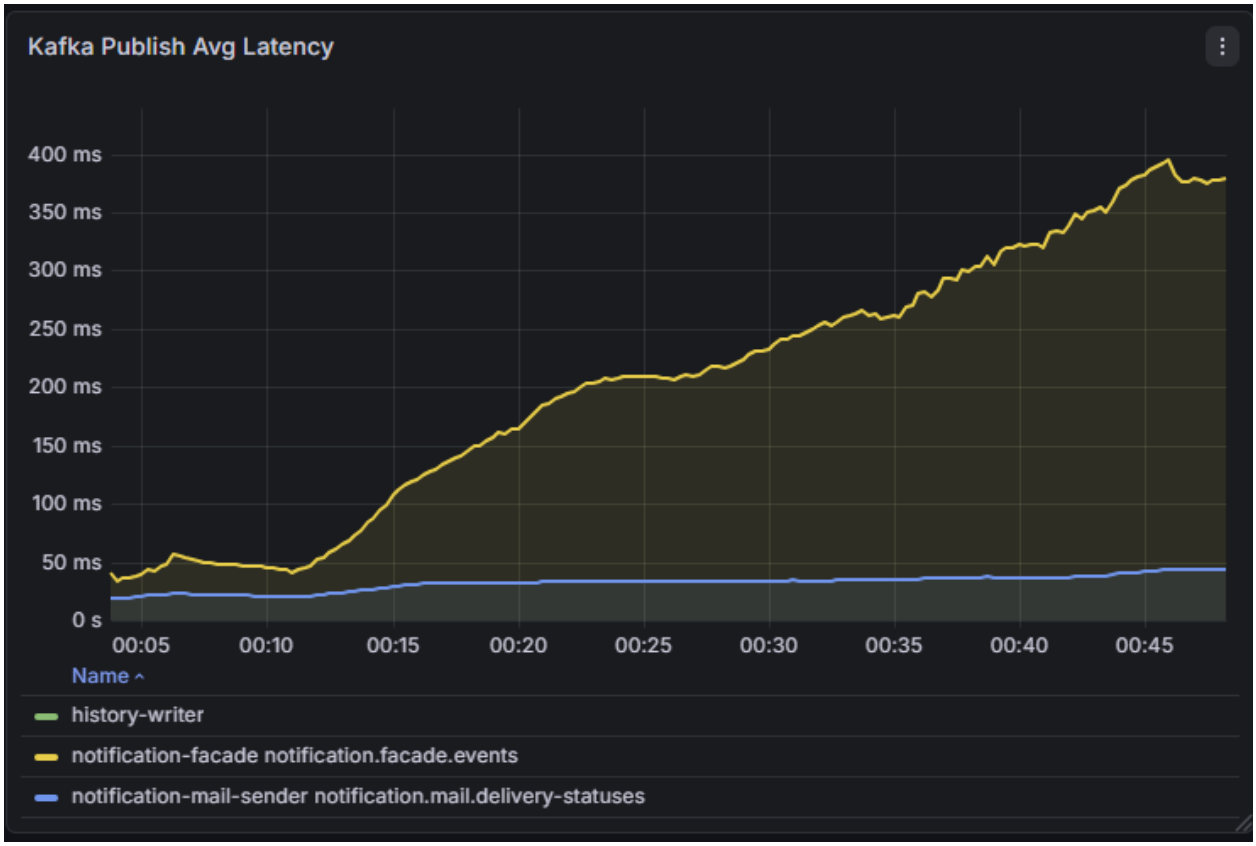


Рисунок Г.3 – Средняя задержка публикации сообщений в Kafka

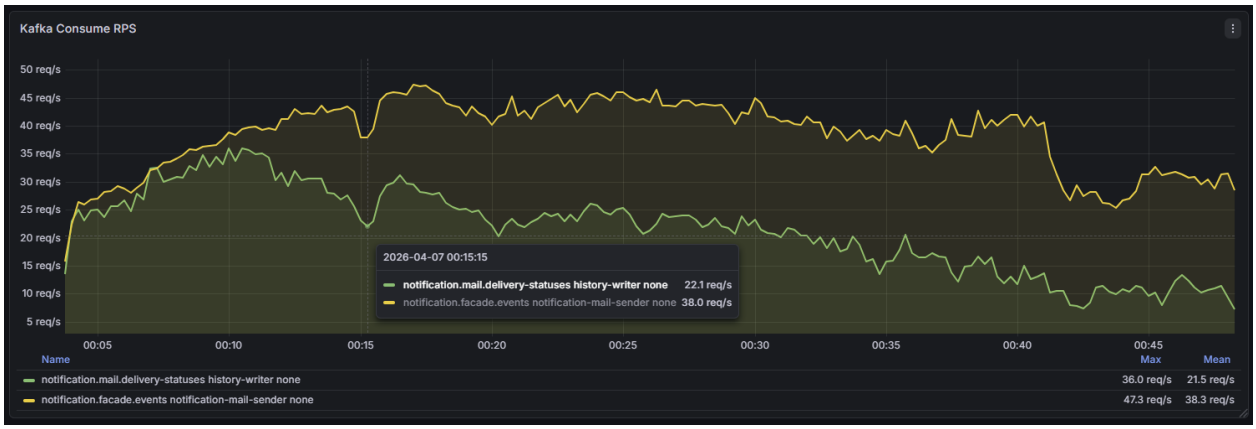


Рисунок Г.4 – Интенсивность потребления Kafka-сообщений на основных потребителях

Интерпретация приведённых артефактов выполняется в основной главе о тестировании. В рамках приложения они сохраняются как подтверждение того, что:

а) входные gRPC-вызовы фасада и профильного сервиса наблюдаемы по интенсивности и задержке;
б) producer-операции Kafka отражаются в метриках времени публикации;
в) consumer-операции sender-сервисов и history-сервиса наблюдаемы как по RPS, так и по времени обработки;
г) метрики пригодны для сопоставления стадий одного и того же потока данных.

ПРИЛОЖЕНИЕ Д
Допущения, ограничения и проверка полноты рукописи

0.24 Проверка полноты обязательных частей

Состав подготовленной рукописи проверен по обязательным элементам ВКР. Результат сведён в таблицу Д.1.

0.25 Критические допущения и ограничения

Допущение: В текущий LaTeX-проект не включены отдельные файлы задания на ВКР и календарного плана. Согласно используемому шаблону института эти документы оформляются как внешние материалы и должны быть приложены к итоговому комплекту работы отдельно.

Допущение: Текущая кодовая база реализует полноценные sender-сервисы только для каналов электронной почты и SMS. Канал push присутствует в модели профиля получателя и может быть развит в дальнейшем, но отдельный push-sender в проекте на момент подготовки отчёта отсутствует.

Допущение: В демонстрационном профиле текущей реализации часть gRPC-сервисов и клиентов работает без обязательной аутентификации, а Kafka и отдельные gRPC-клиенты используют plaintext-транспорт. Для промышленного внедрения требуется включить TLS, сервисную аутентификацию и централизованное управление секретами.

Допущение: Часть адаптеров доставки может работать в тестовом режиме log-only или через логирующий gateway. Это позволяет верифицировать программный пайплайн, но не заменяет отдельную промышленную проверку реальных внешних провайдеров.

Допущение: Экономическая глава выполнена как качественная оценка, поскольку в составе репозитория отсутствуют утверждённые числовые исходные данные, необходимые для полного нормативного калькуляционного расчёта.

0.26 Проверка целостности текста

При доработке рукописи были дополнительно проверены следующие аспекты:

а) единообразие терминов событие, запуск доставки, канальная доставка;
б) отсутствие в основном тексте утверждений о несуществующем push-sender;
в) согласованность описания data flow с фактическими gRPC- и Kafka-контрактами;
г) наличие источников для обзорных и теоретических положений;
д) наличие выводов по каждой основной главе.

Таблица Д.1 – Проверка полноты обязательных частей ВКР

Обязательная часть	**Статус**	**Комментарий**
Титульный лист	Есть	Формируется средствами шаблона diploma.
Реферат	Есть	Подготовлен как отдельный файл главы.
Содержание	Есть	Формируется автоматически.
Введение	Есть	Содержит актуальность, цель, задачи, объект и предмет.
Аналитическая глава	Есть	Включает анализ предметной области и обоснование выбора технологий.
Постановка задачи и требования	Есть	Включены в аналитическую главу отдельным подразделом.
Проектирование архитектуры	Есть	Описаны сервисы, потоки данных, надёжность и безопасность.
Моделирование данных	Есть	Описано в главе реализации и приложении.
Описание API/контрактов	Есть	Вынесено в главу реализации и приложение с контрактами сервисного взаимодействия.
Реализация	Есть	Отражены реальные сервисы и фактический кодовый пайплайн.
Тестирование	Есть	Описаны методика, критерии успешности и артефакты наблюдаемости.
Экономическая часть / безопасность	Есть	Выполнена качественная оценка и анализ рисков.
Заключение	Есть	Сформулированы результаты и направления развития.
Список источников	Есть	Оформляется через biblatex.
Приложения	Есть	Добавлены приложения по контрактам, Kafka, данным и наблюдаемости.
Задание на ВКР	Требует внешнего документа	См. допущение ниже.
Календарный план	Требует внешнего документа	См. допущение ниже.